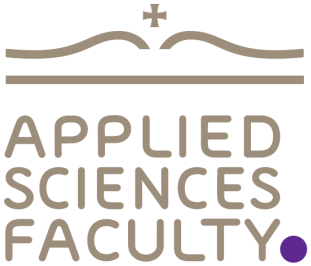
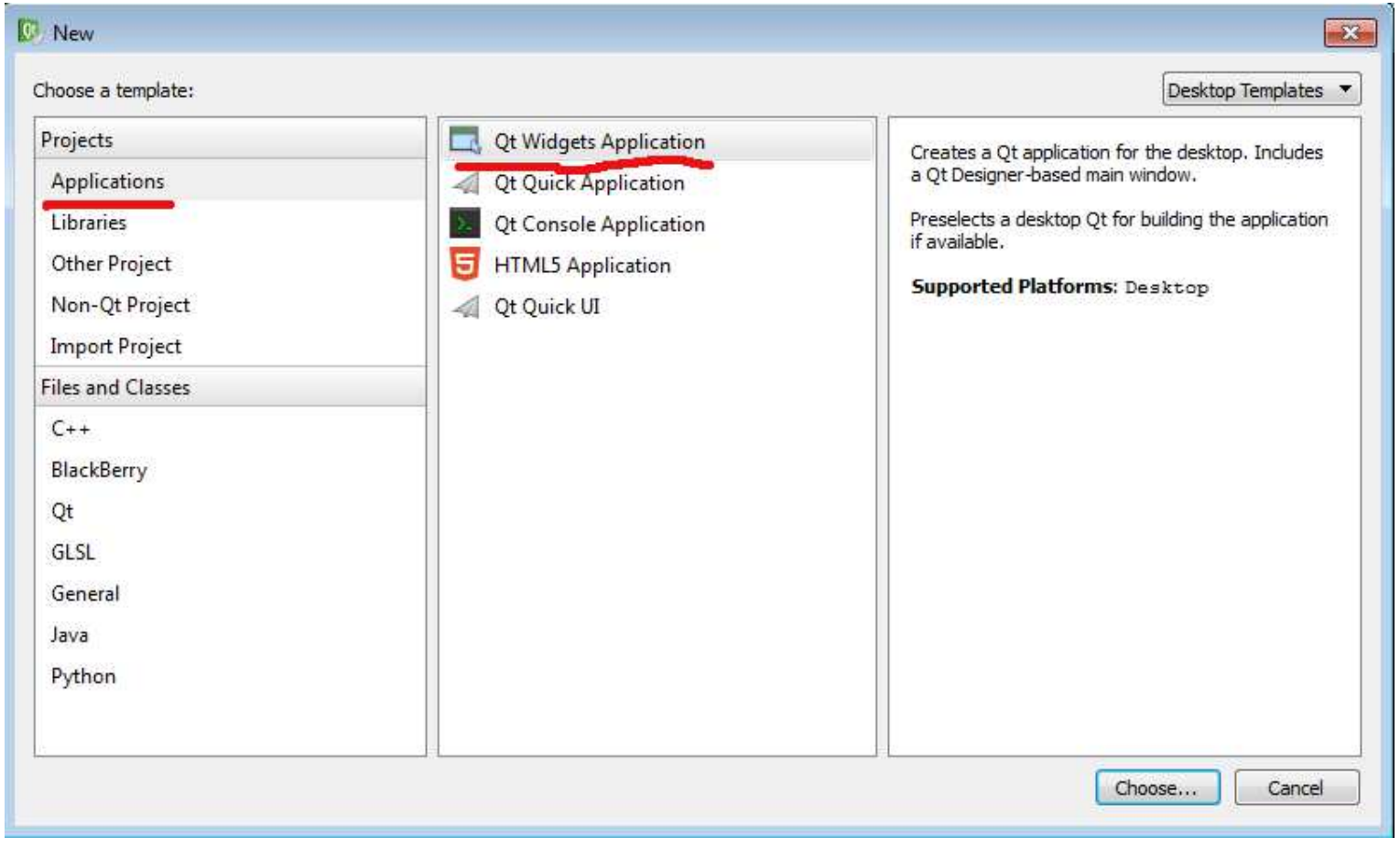
C++ — додаток. Qt. Lecture 5 — текстовий редактор та все йому необхідне





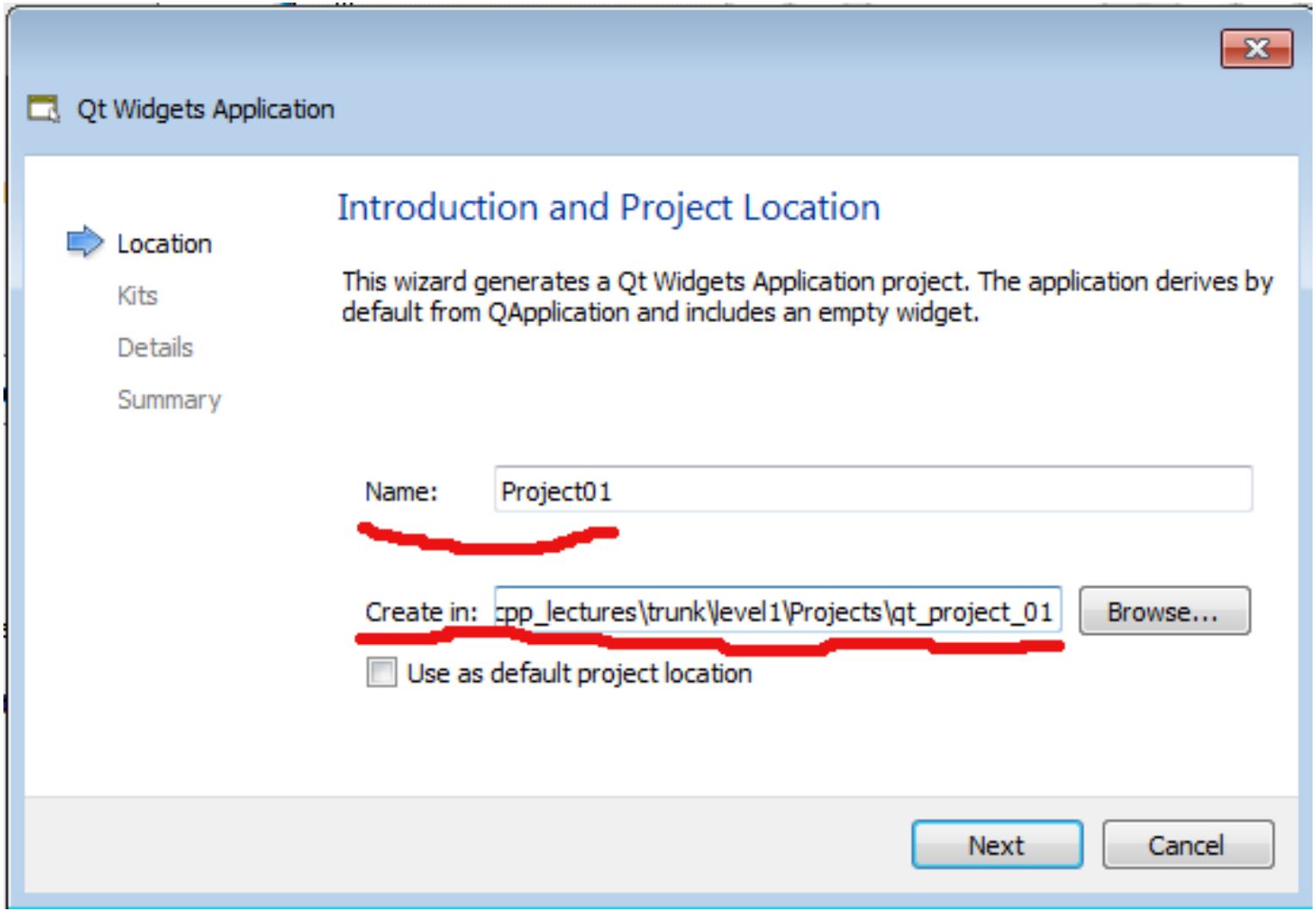
©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Створювати проекти повністю вручну незручно. Qt Creator передбачає можливість візуального редагування[1]. Щоб створити відповідний проект, вибираємо "Qt Widgets Application":

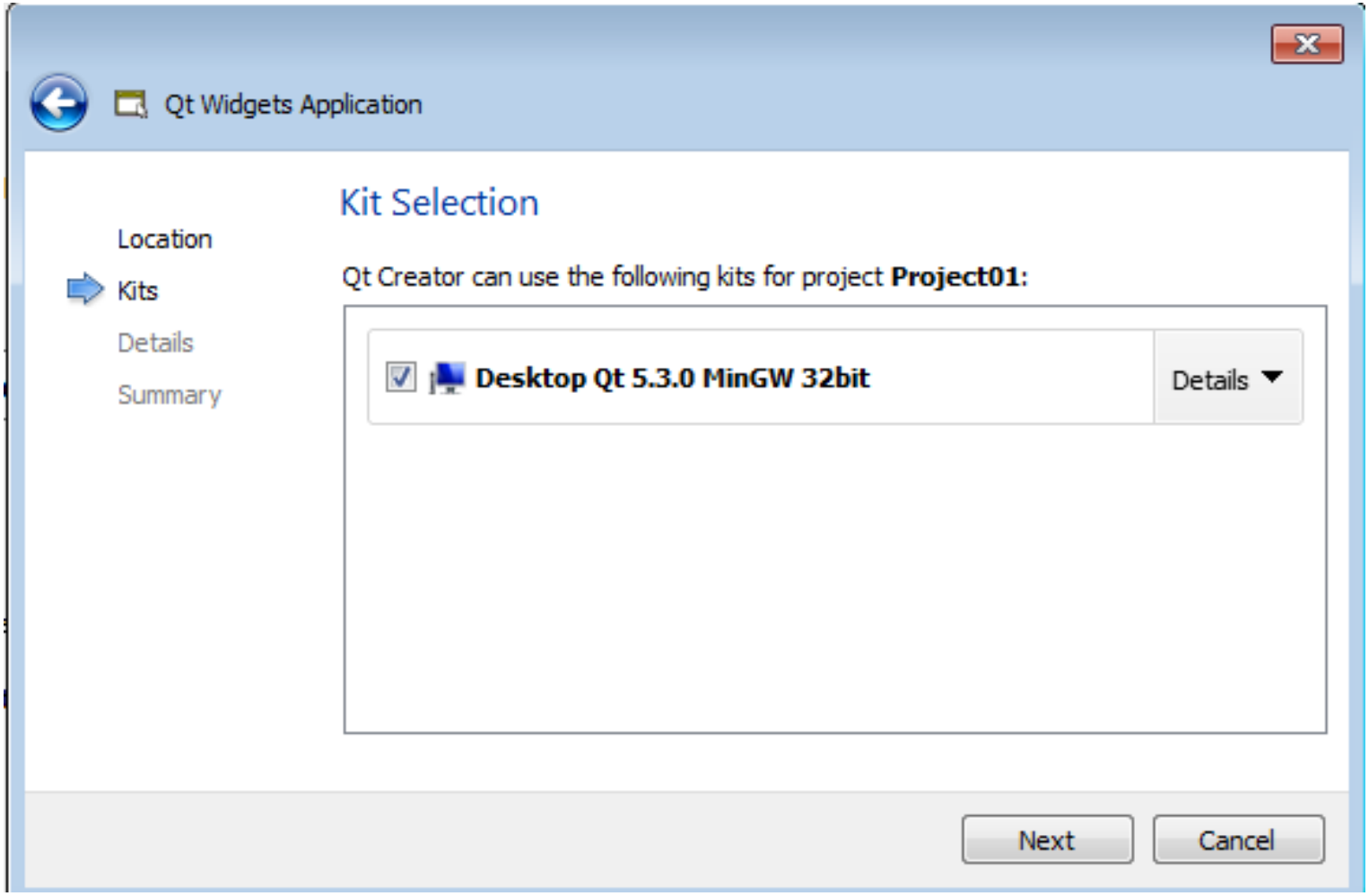


[1] Delpi ще хтось пам'ятає? :-)

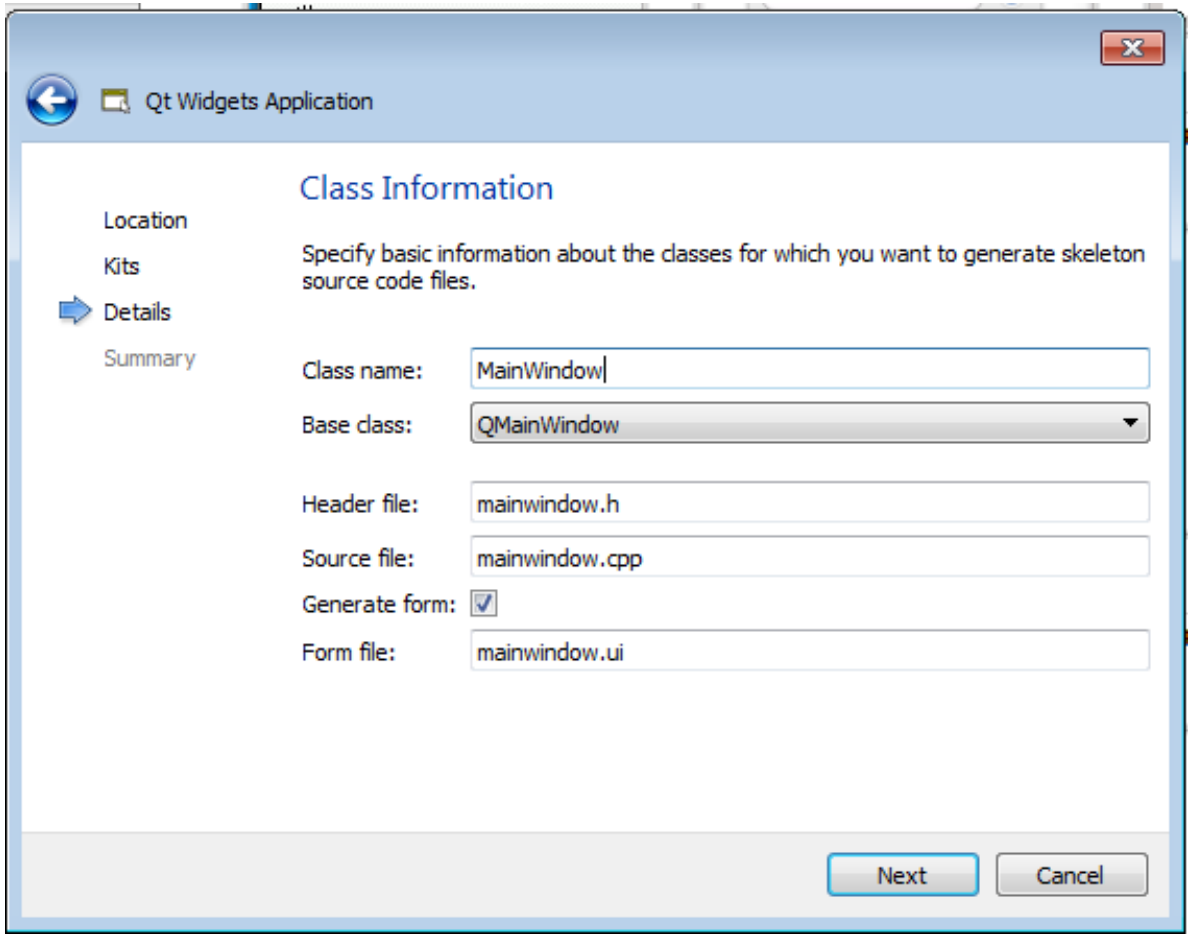
Вибираємо ім'я проекту та шлях до нього:



Вибираємо комплект Qt, який використовувати. В даному випадку присутній лише один, однак їх може бути декілька — для різних компіляторів, різної розрядності, різних архітектур, тощо.

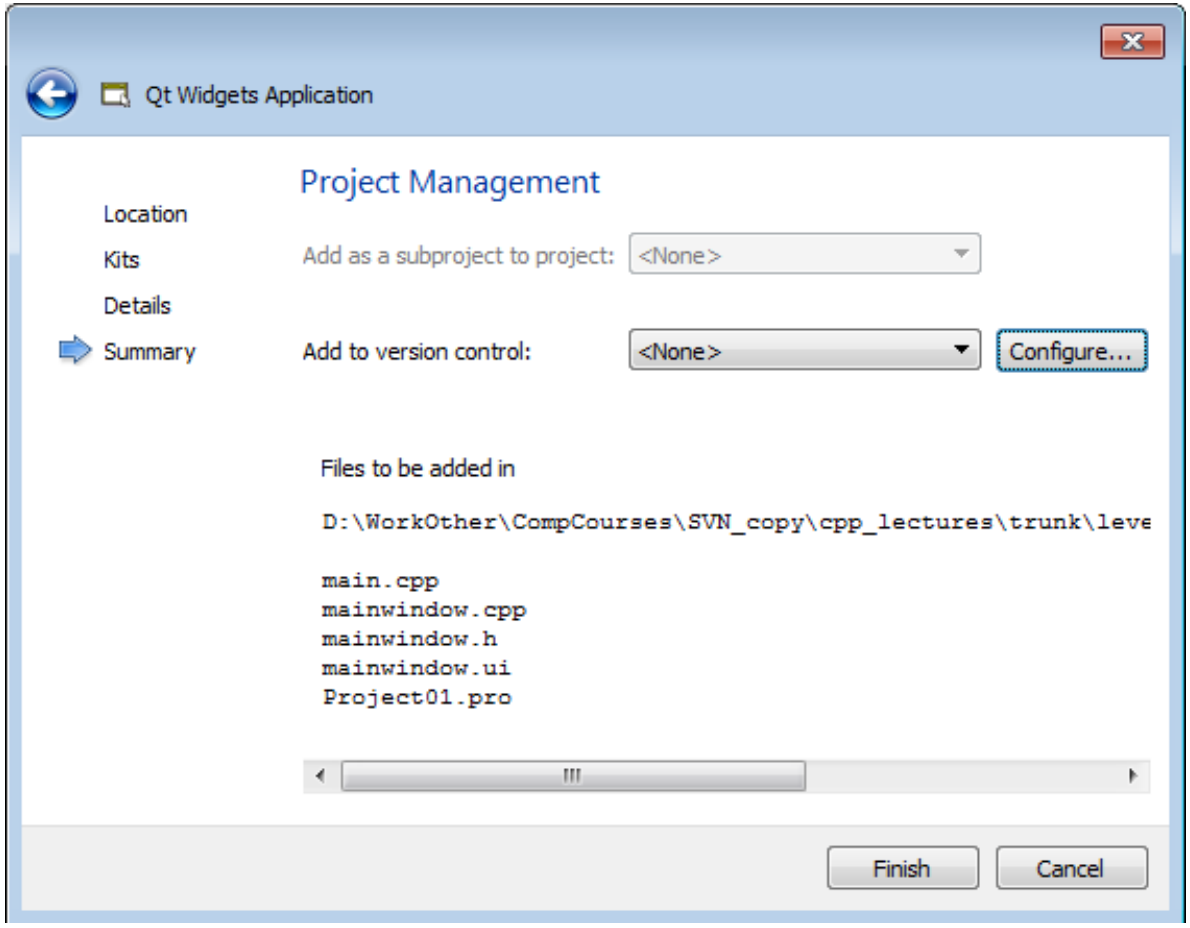


На наступному етапі можна вибрати деякі важливі параметри. Зокрема, який клас буде предком головного віджету програми. По замовчуванню це QMainWindow. Можна також вибрати QWidget і QDialog. Середовище автоматично створює клас головного widget-у програми, відповідний файл заголовків та .cpp-файл:



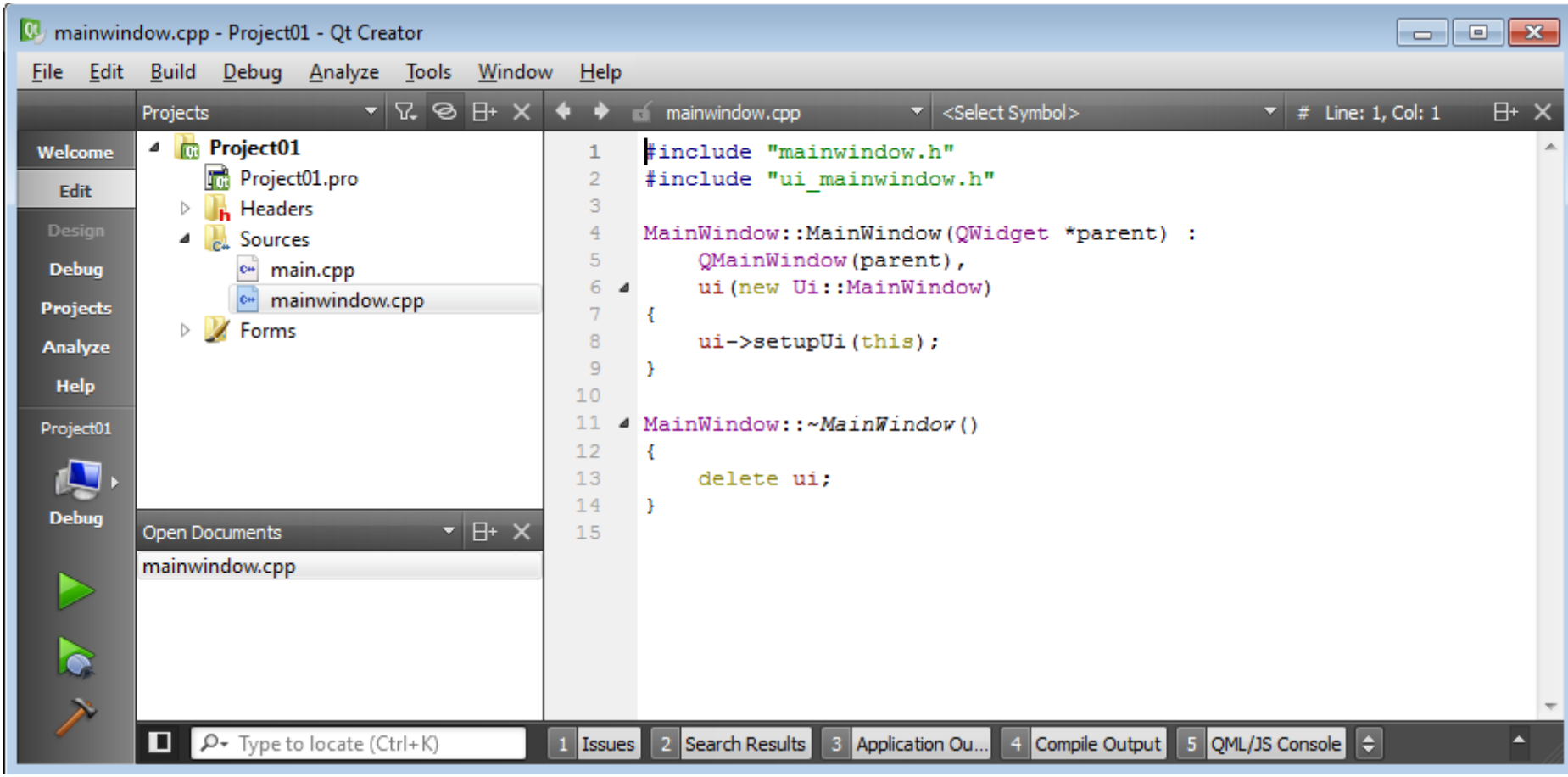
На цьому етапі можна також задати імена класу, файлів. Якщо не забороняти, буде згенеровано файл форм, .ui, який містить інформацію для (візуального) редактора форм. User Interface Compiler (uic) компілює цей файл, створюючи відповідні заголовки, .h, та .cpp файли.

На останньому кроці можна також підключити використання з проектом тієї чи іншої системи контролю версій:



Нарешті тиснемо Finish і отримуємо готовий проект.

Створений проект



Текстовий редактор

Звичайно, не позбавлено певної цікавості відтворити створення того ж проекту, що раніше — "Hello, image", засобами Qt Designer. Однак, це не дуже продуктивно. Тому цього разу вирішимо іншу класичну задачу — створимо простий текстовий редактор.

Аналіз заготовки

- Головна програма менш-більш звична:

```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();

    return a.exec();
}
```

- Файл mainwindow.h цікавіший:

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>

namespace Ui {
class MainWindow;
}

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = 0);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
};

#endif // MAINWINDOW_H
```

- Код:

```
namespace Ui {
class MainWindow;
}
```

є forward-оголошенням класу, який реалізовує графічну частину нашої програми. Клас цей оголошується у просторі імен Ui, стандартному для коду, згенерованого uic з форми (.ui).

- Код для MOC — Q_OBJECT, та конструктор із деструктором менш-більш звичні. Конструктор отримує вказівник на "батьківський" клас. По замовчуванню — 0, нульвоий вказівник, який означає, що вікно не має предків — є головним, "самостійним".
- Цікавішим є приватне поле:

```
private:
    Ui::MainWindow *ui;
```

який оголошує вказівник на ui-клас, який відповідає за візуальну частину інтерфейсу, та генерується автоматично.

- Заготовка mainwindow.cpp :

```
#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

- Файл включає оголошення класу, яке ми розглянули вище:

  #include "mainwindow.h"

- та автоматично згенеровані оголошення для візуального інтерфейсу:

  #include "ui_mainwindow.h"

- Конструктор:

```
MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}
```

  ініціалізує базовий клас, QMainWindow, створює екземпляр візуального інтерфейсу ("ui(new Ui::MainWindow)") та ініціалізує його.

- Деструктор видаляє UI:

```
MainWindow::~MainWindow()
{
    delete ui;
}
```

Файл проекту виглядатиме так:

```
QT       += core gui

greaterThan(QT_MAJOR_VERSION, 4): QT += widgets

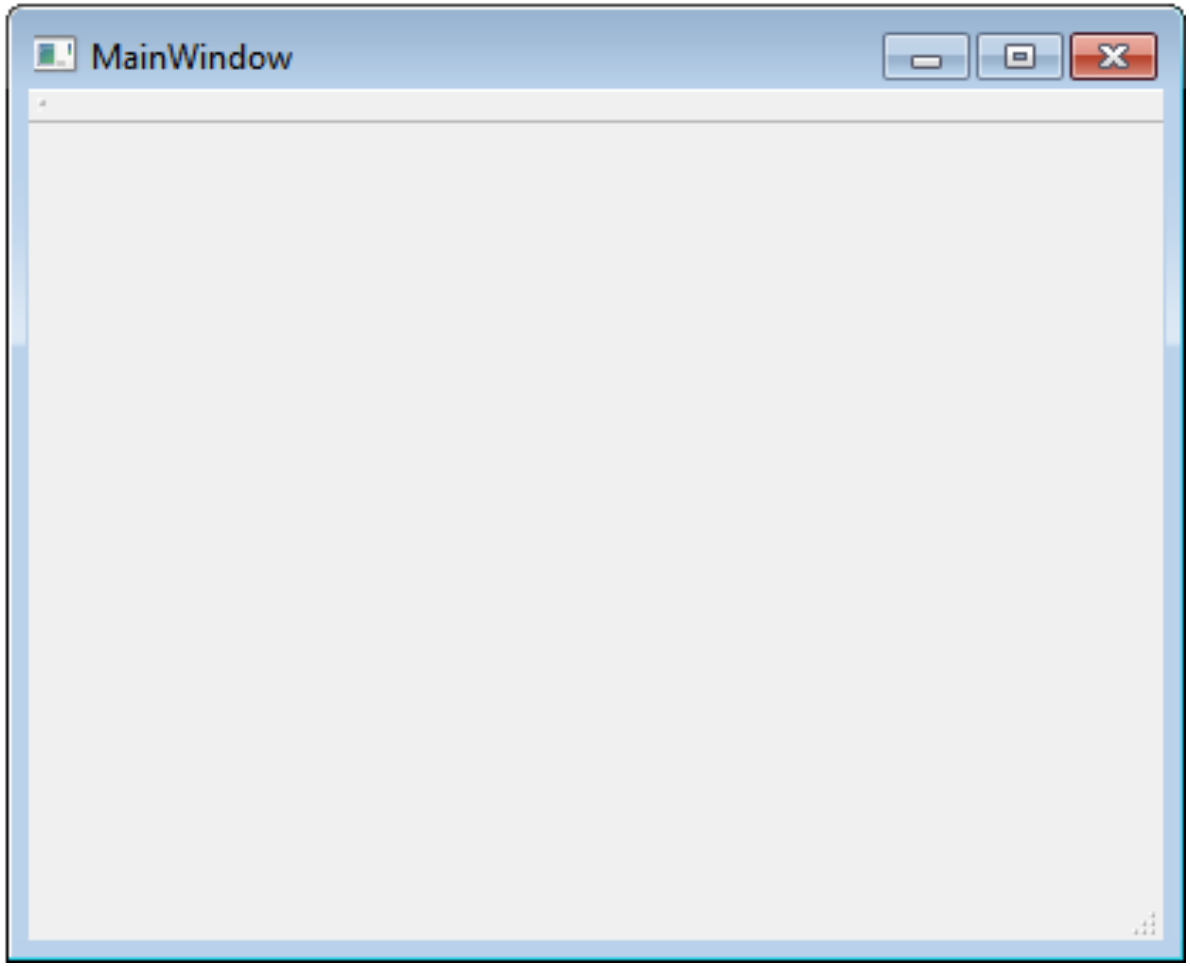
TARGET = qtl06_ui_editor
TEMPLATE = app


SOURCES += main.cpp\
        mainwindow.cpp

HEADERS  += mainwindow.h

FORMS    += mainwindow.ui
```

А програма так:

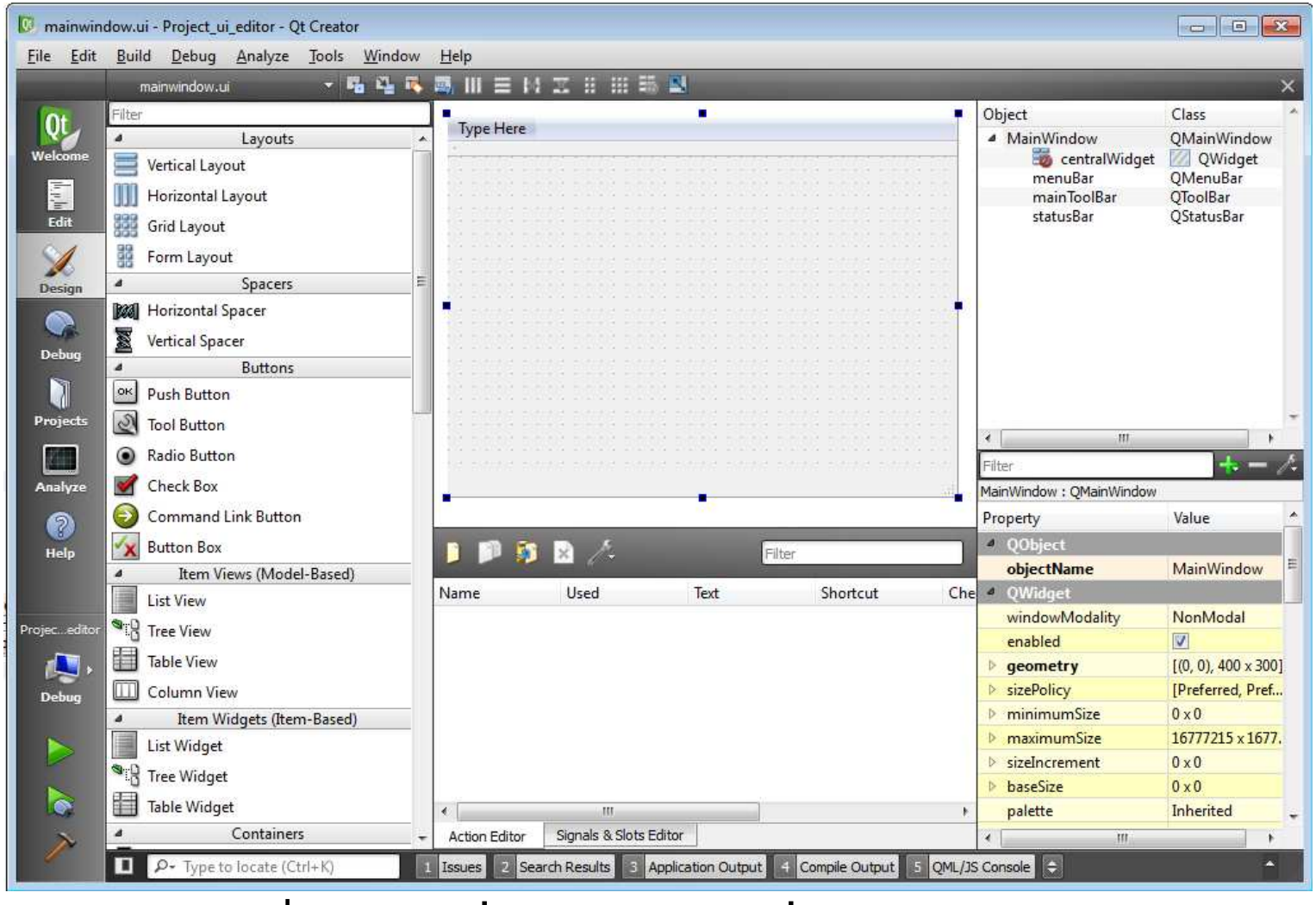


Почнемо з того, що намалюємо інтерфейс нашої програми

Вікно редактора включатиме наступні компоненти:

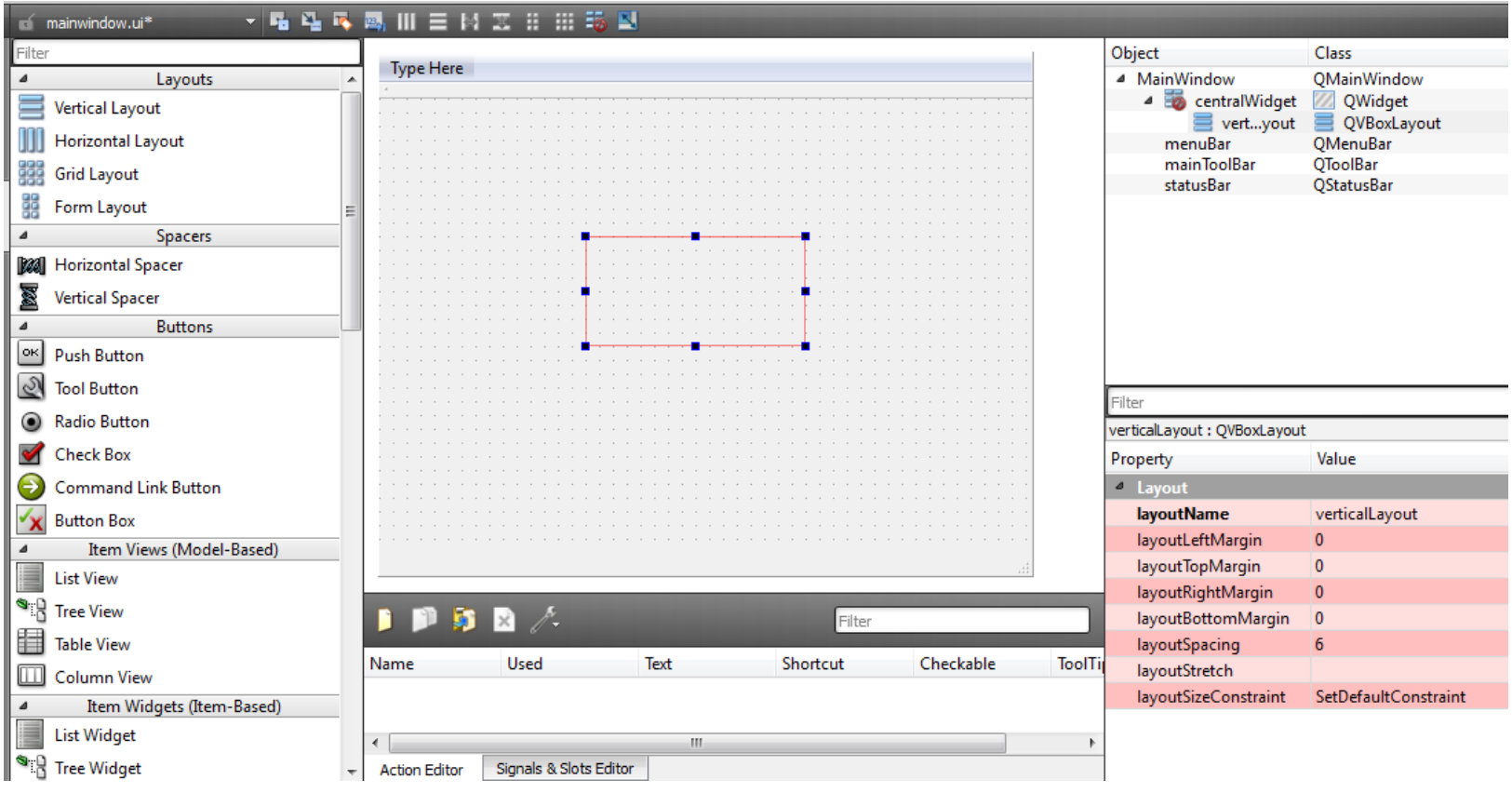
- Поле для редагування тексту.
- Меню з пунктами File (New, Open, Save, Save as), Edit (Copy, Paste, Cut, Delete, Find, Replace, Undo, Redo), Help (About).
- Відповідні панелі інструментів.
- Рядок статусу, який міститиме ім'я файлу, з підказкою, що показуватиме повний шлях, поточний рядок та стовпець у файлі і кількість символів у ньому.

Щоб почати редагування, відкриємо файл форми, `mainwindow.ui` — запуститься візуальний редактор:

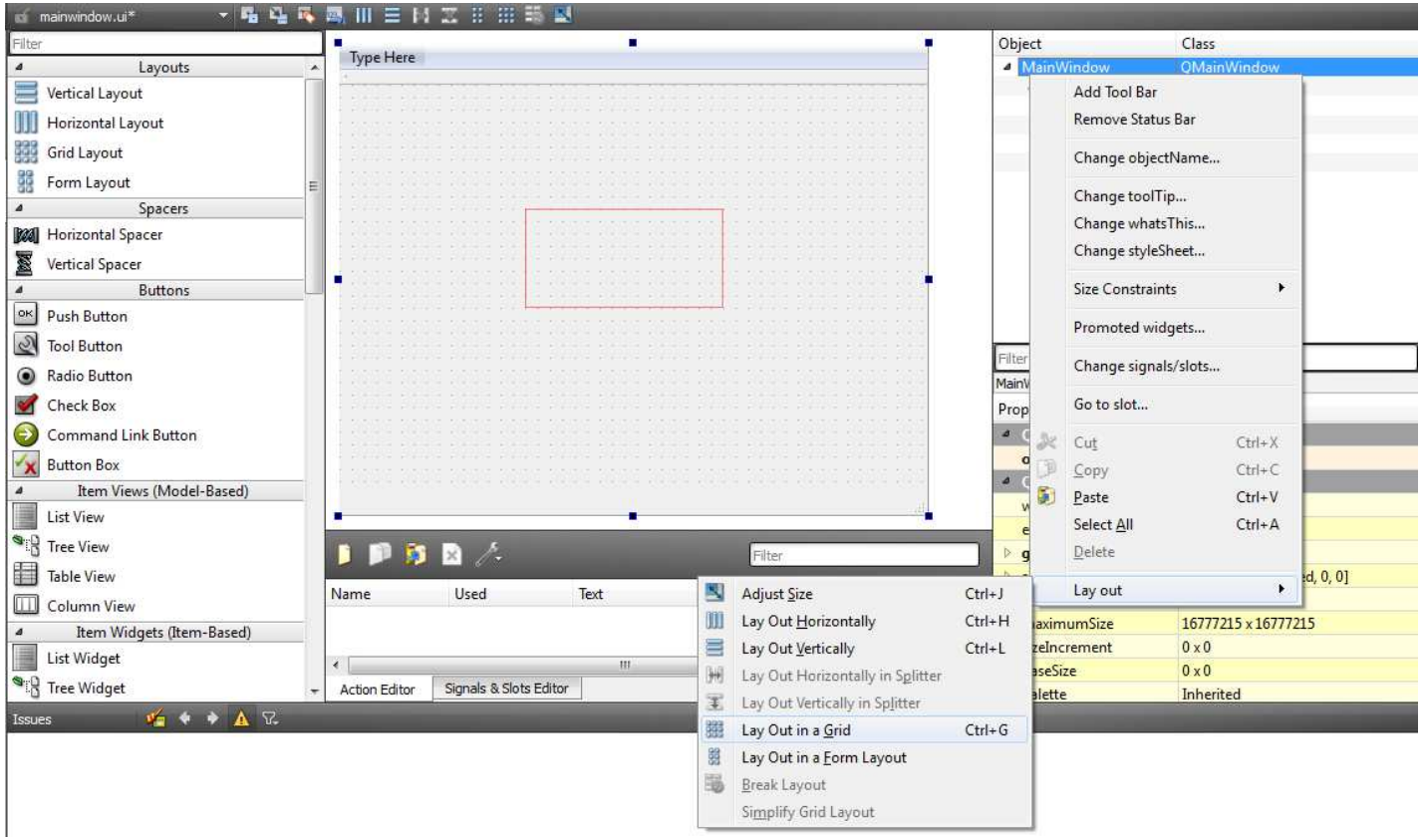


По центру — форма, ліворуч — палітра компонент, праворуч — список об'єктів на них зверху та їх властивості внизу.

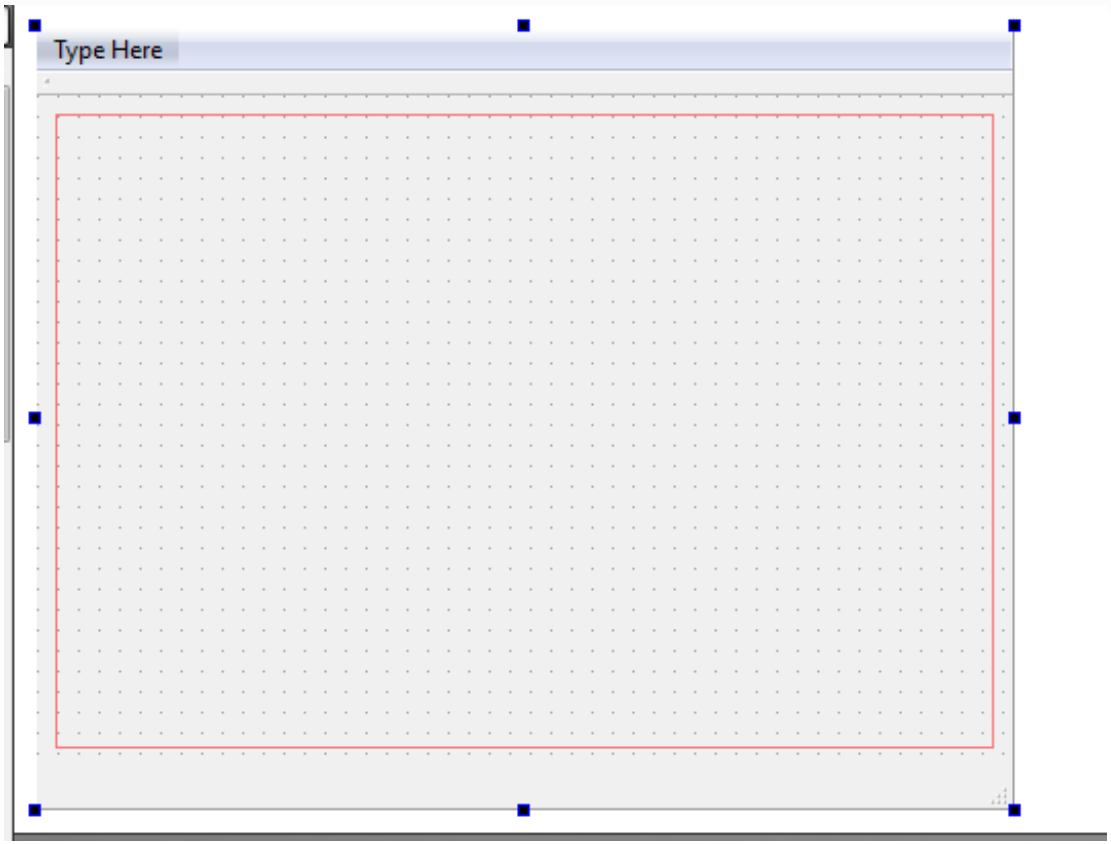
- Нам би хотілося, щоб текстове поле, QTextEdit, змінювало свій розмір разом із вікном. Реалізація цього засобами Qt Designer вимагає трішки не очевидної магії.
- Спочатку виберіть ліворуч один із менеджерів розташування (layout managers), наприклад вже знайомий QVBoxLayout. Перетягніть його на форму:



Тепер виділіть MainWindow в Object Inspector праворуч зверху, натисніть на ньому праву кнопку миші, ”Lay out”, ”Lay Out in a Grid”:

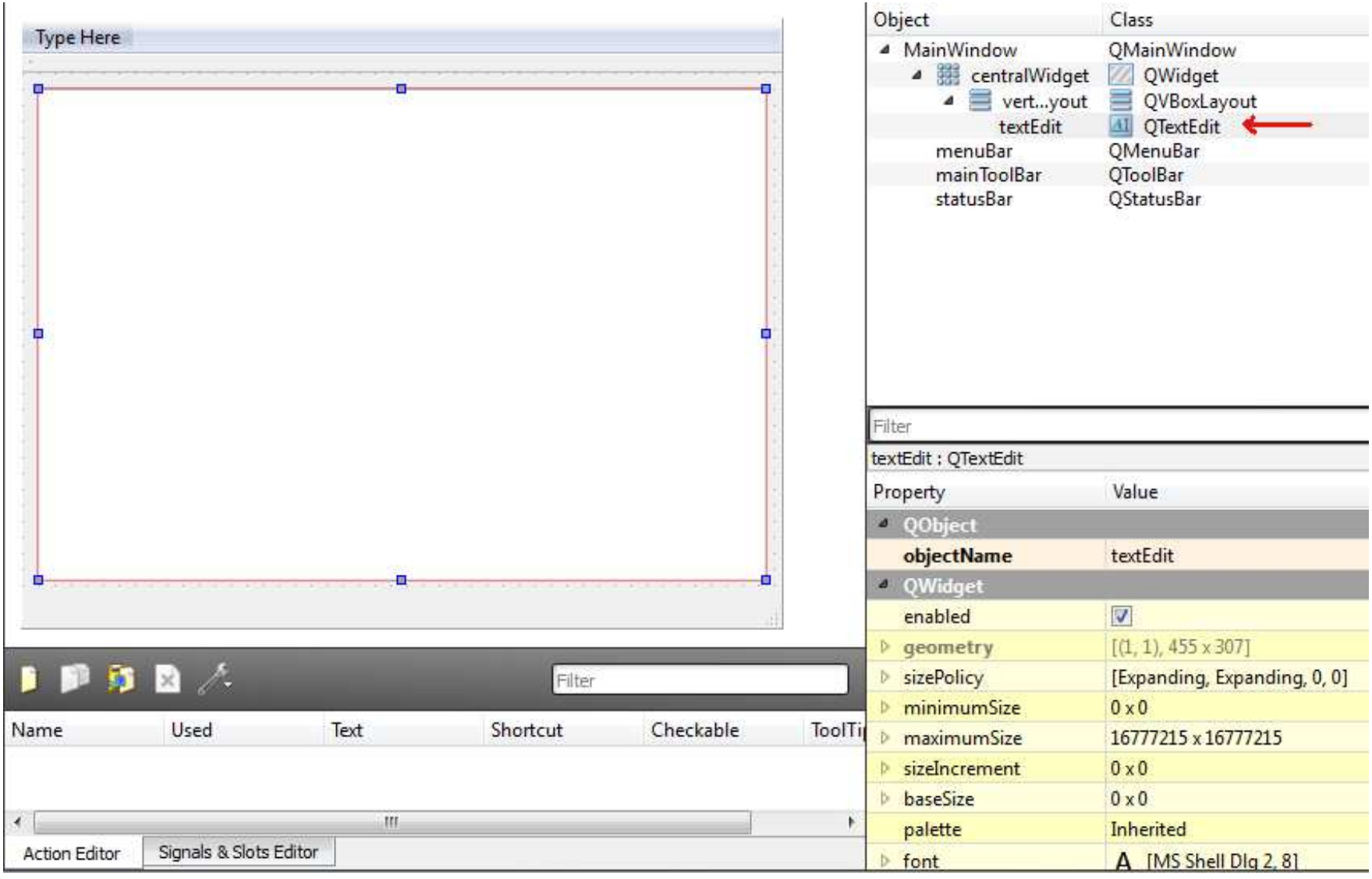


Отримаємо таку форму:

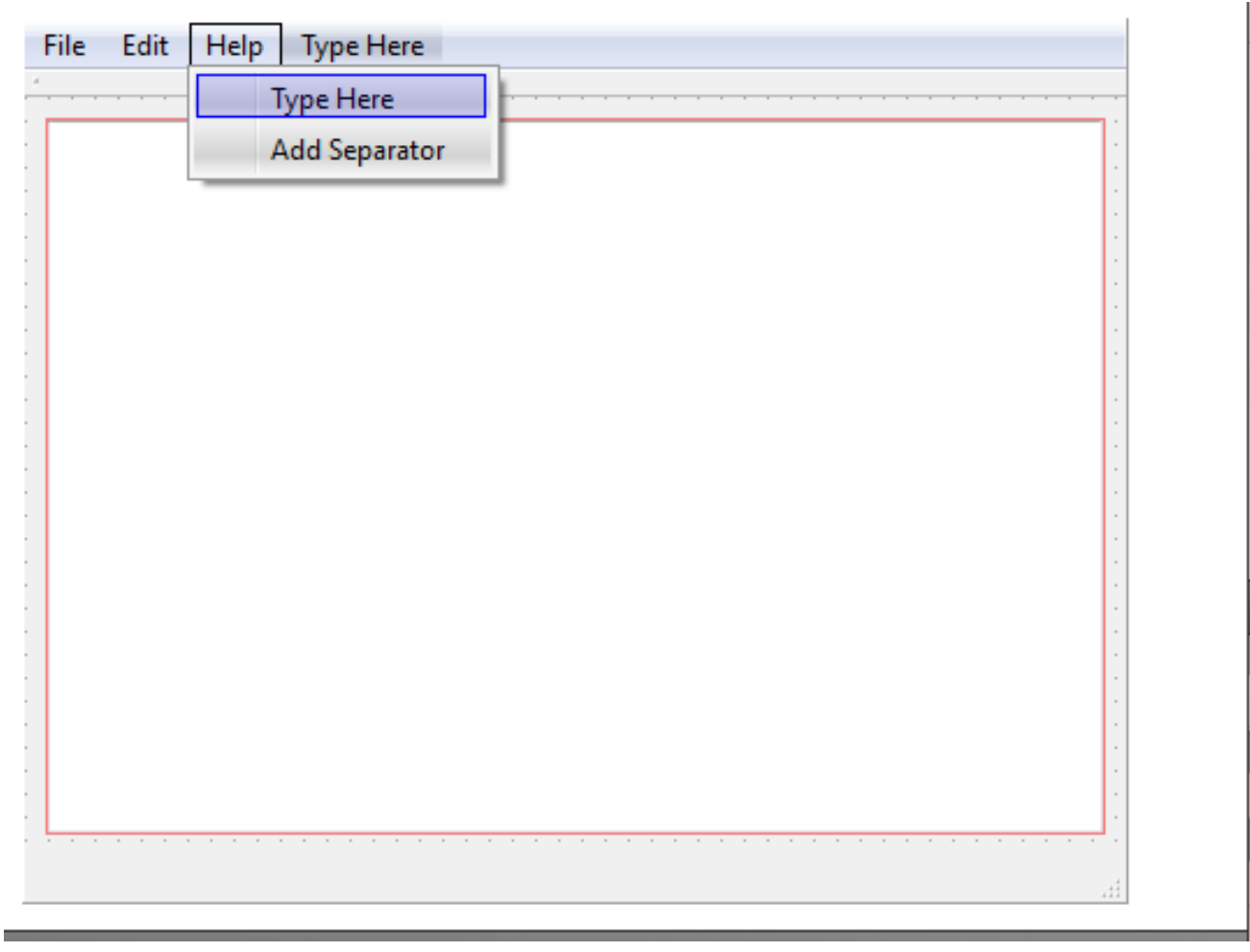


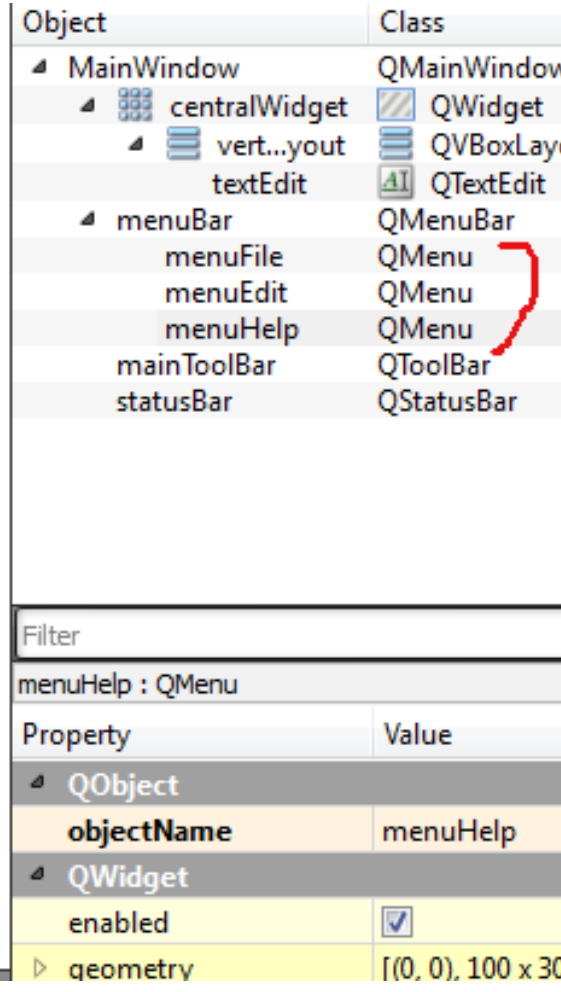
Тепер layout буде змінювати свій розмір разом із вікном.
Можна тепер до цього layout додати QTextEdit.
На жаль, якщо спершу додати QTextEdit, а потім зроити оце "Lay Out in a Grid", QTextEdit буде складно переконати займати все виділене поле і коректно змінювати свій розмір. Тому краще робити саме в такому порядку.
Можливо, це просто баг моєї версії QtCreator, звичайно...

Перетягнувши QTextEdit, отримаємо таке:

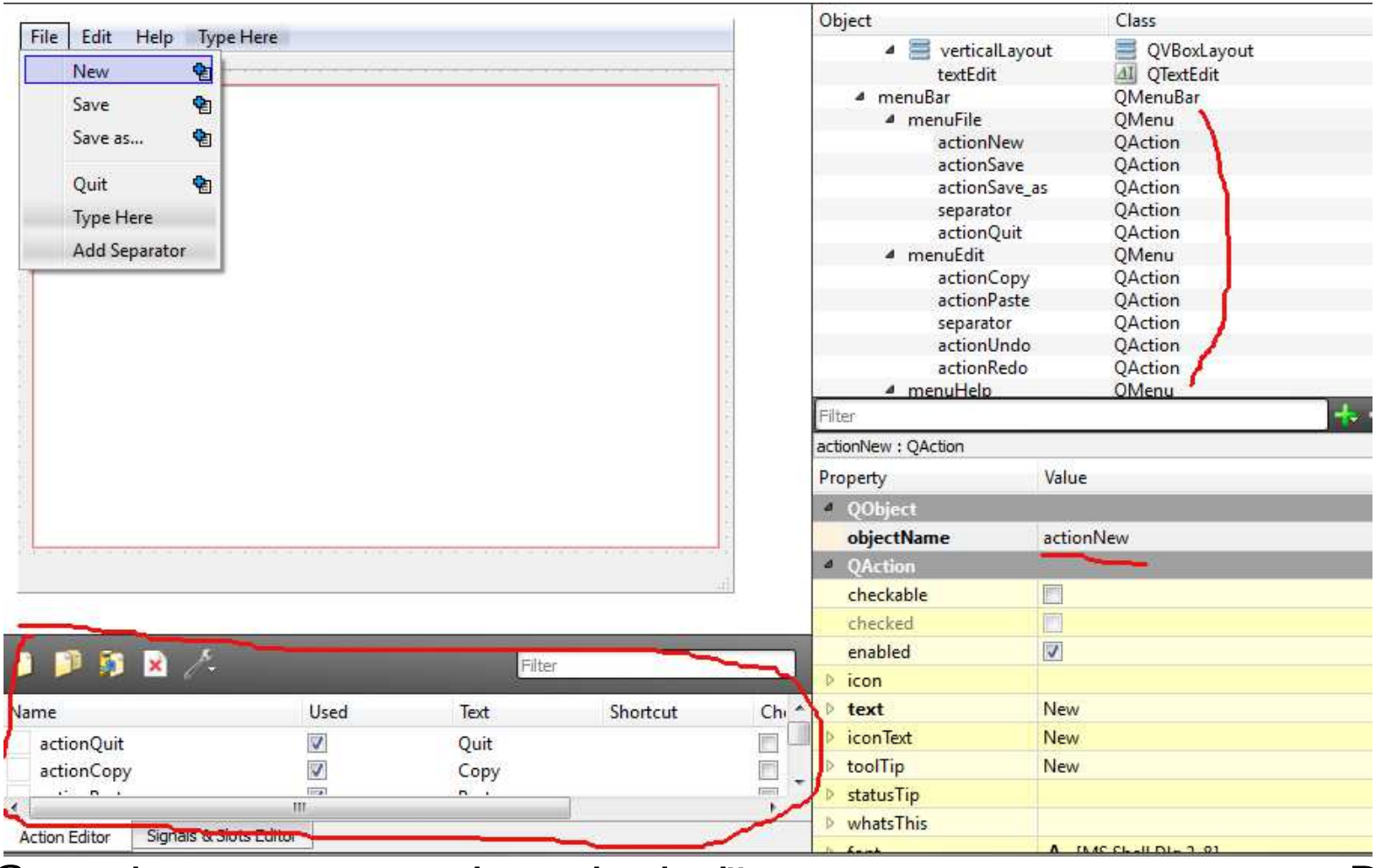


Тепер, там де, ліворуч зверху на формі, пише Type Here, вводимо заплановані пункти меню:



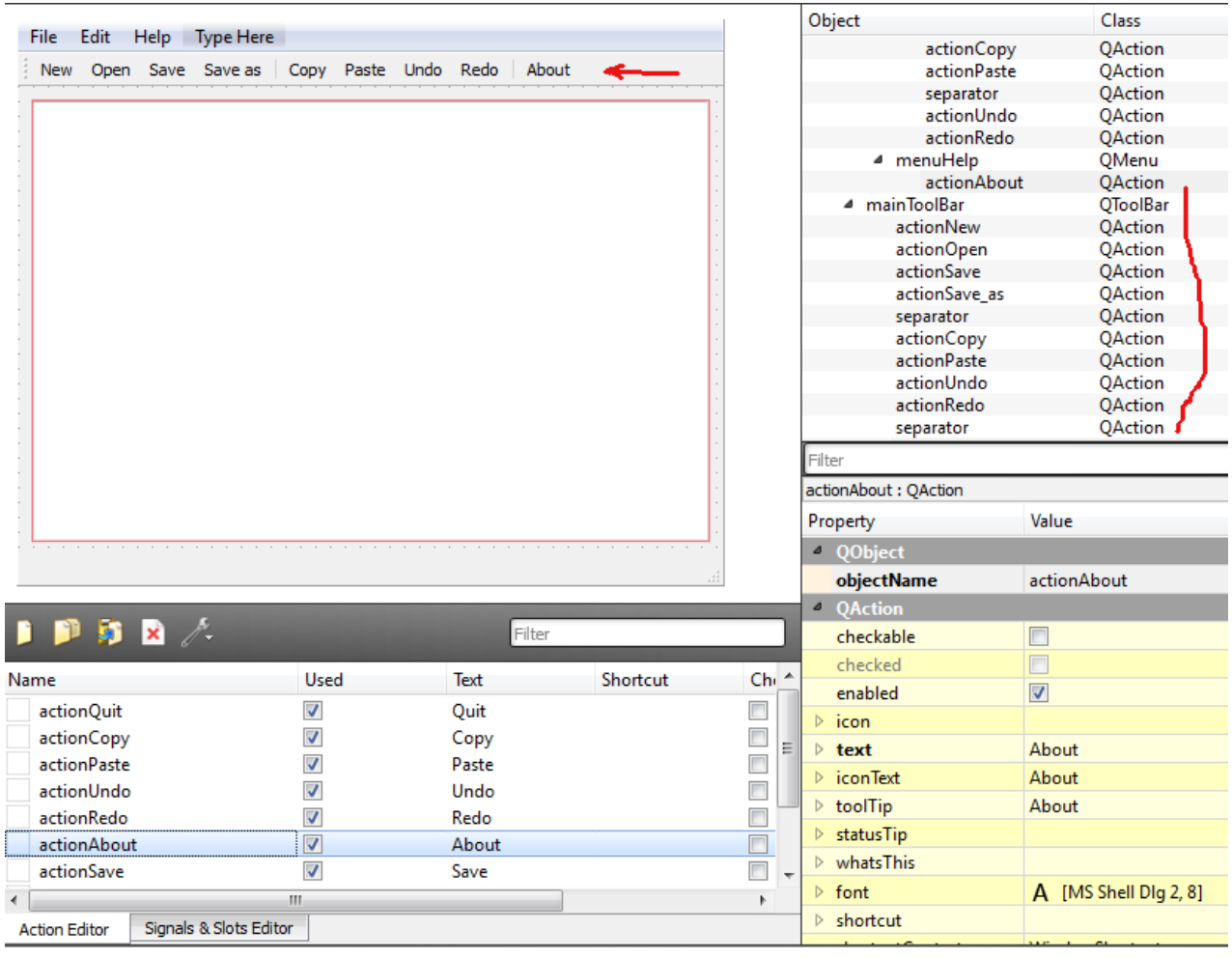


В кожному з них створюємо необхідні підпункти:



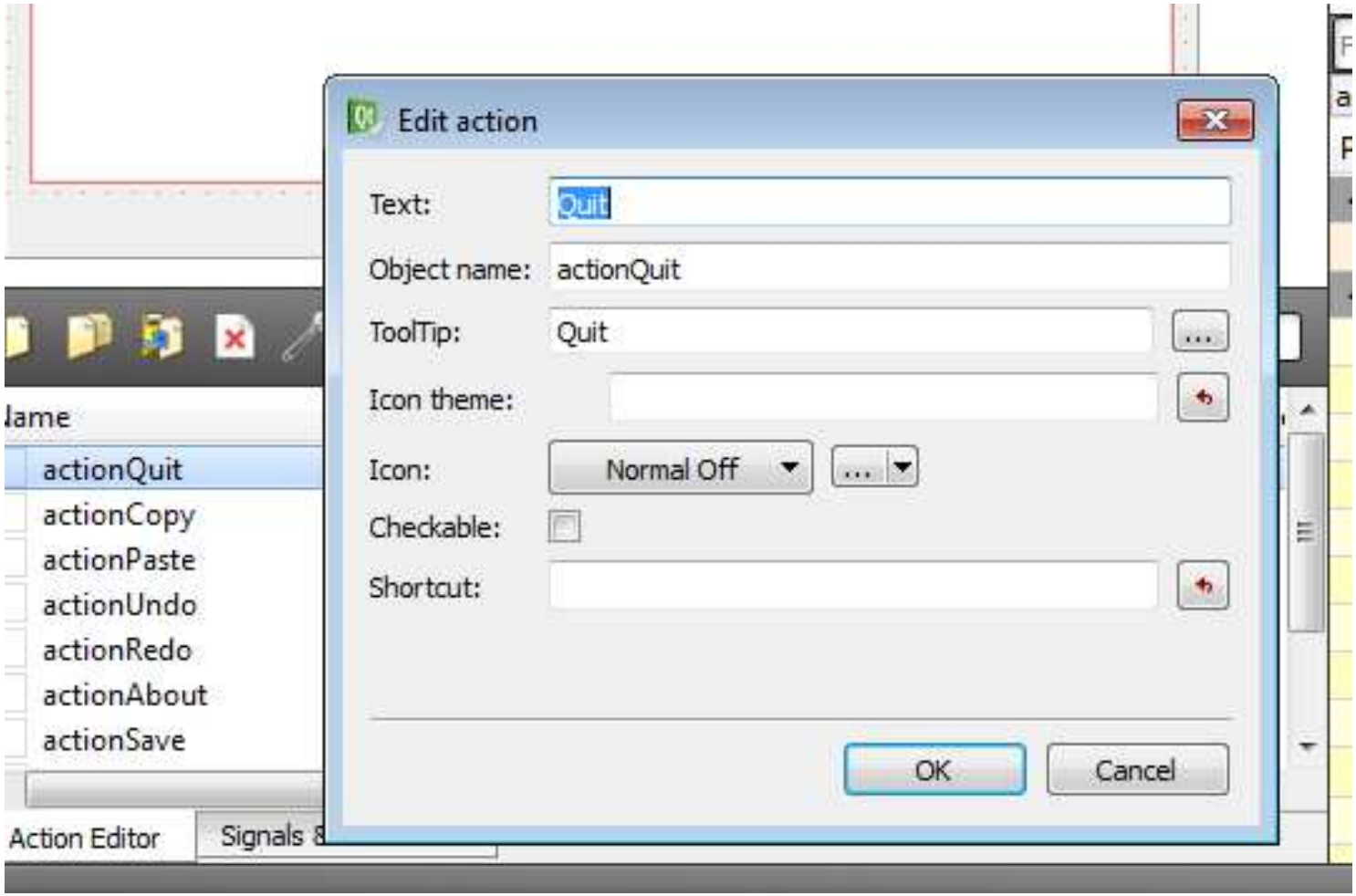
Зверніть увагу, що відповідні дії додаються автоматично. В редакторі дій (внизу, обведено), можна ці дії редагувати. Цим ми займемося пізніше.

Щоб наповнити панель інструментів, перетягуємо на неї (вузька горизонтальна, сіра смужка під меню) дії з редактора дій:



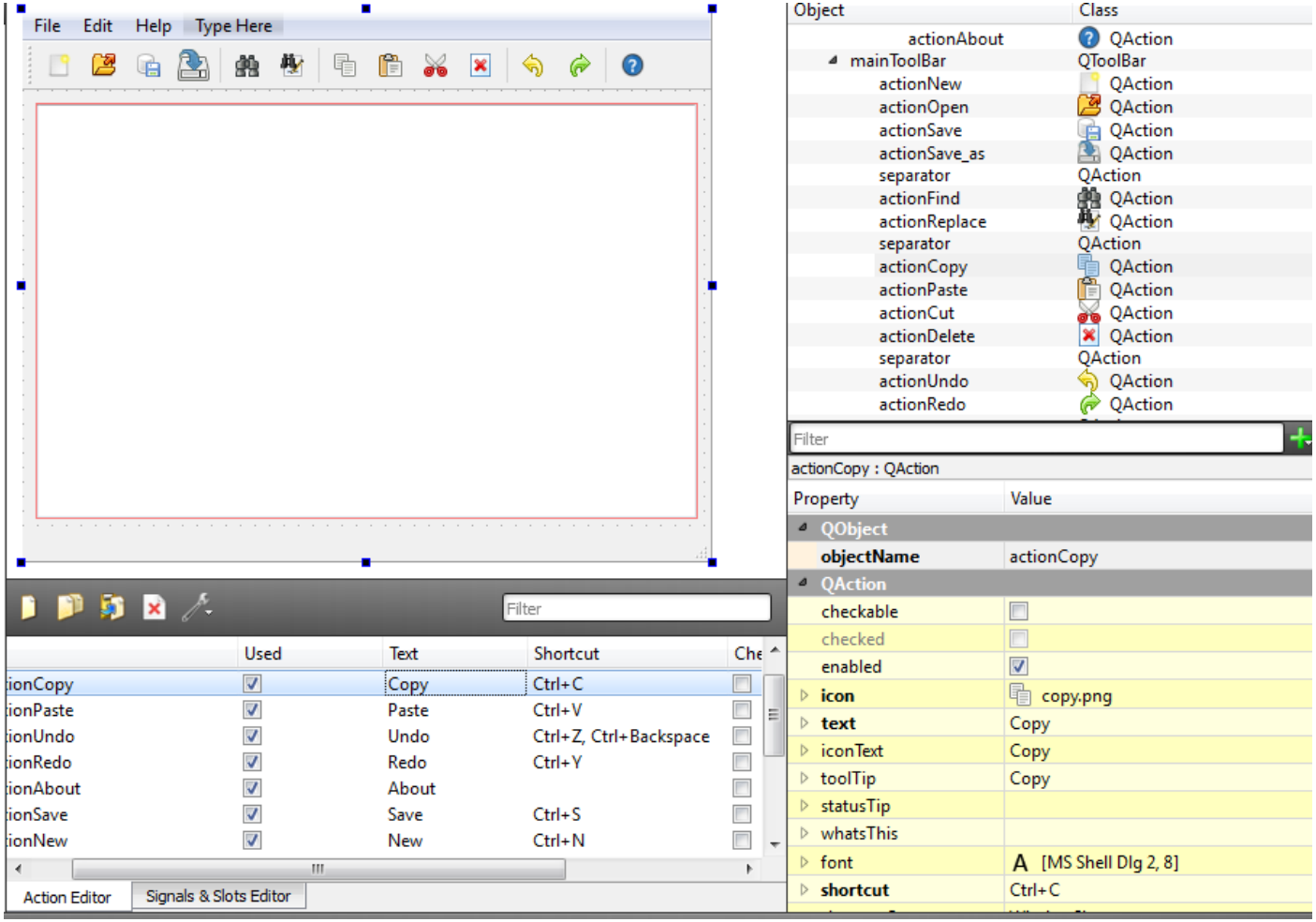
Сепаратор можна додати, через контекстне меню для mainToolBar в редакторі об'єктів, праворуч зверху.

Поки панель інструментів містить виключно тексти. Щоб додати іконки до дій, скористаємося редактором дій — з контекстного меню дії вибрати "Edit...", отримаємо віконечко редагування:



В полі Icon вибираємо іконку, з файлу чи ресурсів. Звичайно, краще спершу додати всі іконки до ресурсів.

Розставивши іконки та, при нагоді, клавіатурні комбінації (shortcuts), отримаємо такий вигляд:



Для редагування слід клікнути, в редакторі дій, на полі Shortcut і натиснути відповідну клавіатурну комбінацію, можна декілька. Наступну дію слід виконати мишкою — клавіша може бути сприйнята, як shortcut.

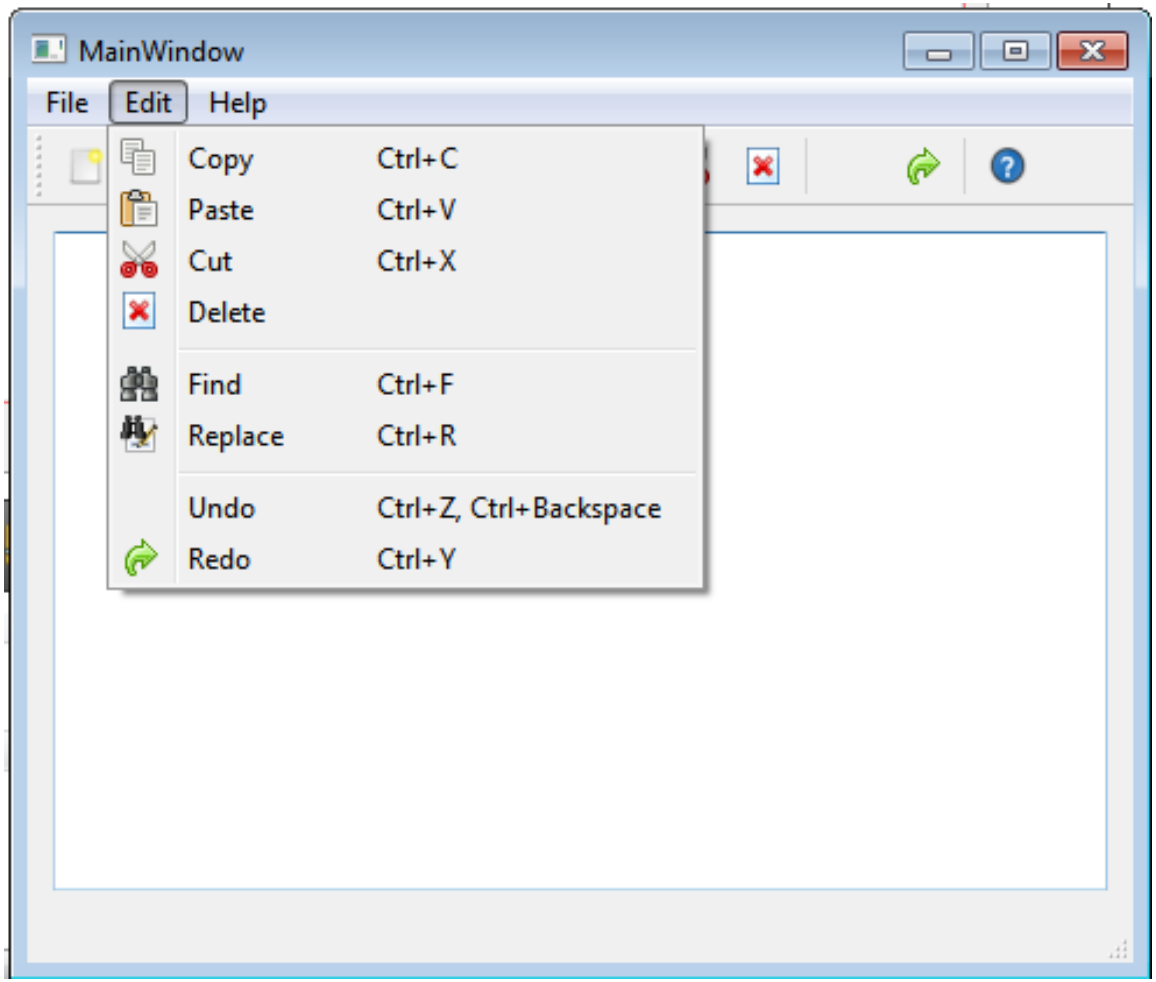
Залишається заселити Statusbar

На жаль, додати QLabel до statusBar засобами Qt Designer, (станом на версію 5.3) неможливо. Тому ми зробимо це безпосередньо в коді.

Перш ніж рухатися далі, рекомендую проглянути в редакторі згенерований для нас mainwindow.ui. Фрагмент з нього:

```
<widget class="QWidget" name="centralWidget">
 <layout class="QGridLayout" name="gridLayout">
  <item row="0" column="0">
   <layout class="QVBoxLayout" name="verticalLayout">
    <item>
     <widget class="QTextEdit" name="textEdit"/>
    </item>
   </layout>
  </item>
 </layout>
</widget>
```

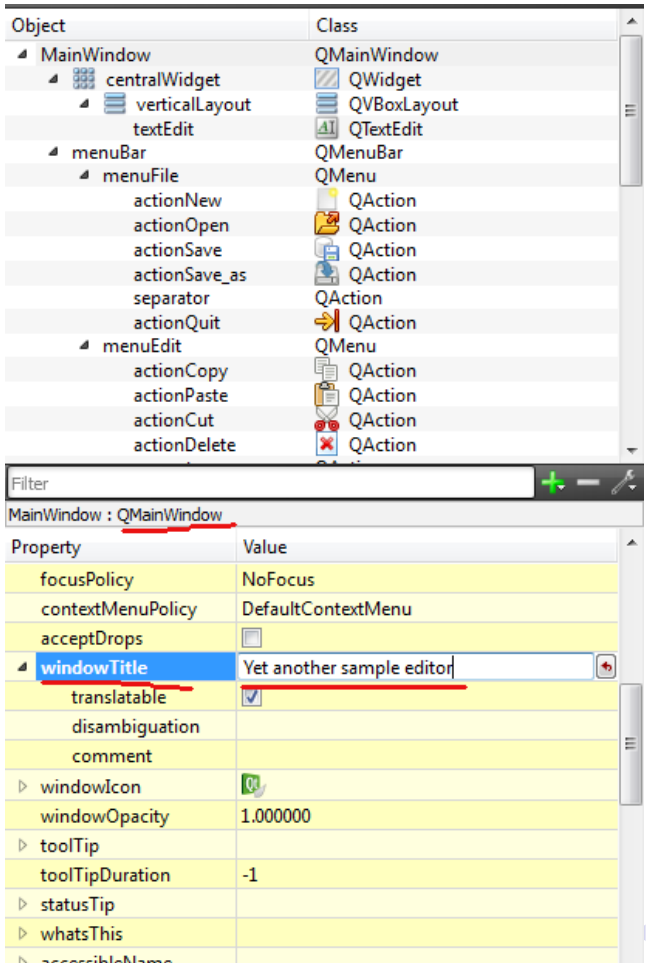
Наша програма тепер виглядає так:



Однак, вона має два мінуси. Один дрібний — вікно називається "MainWindow", інший великий — **вона поки нічого не вміє робити**.

Редагування властивостей

- Назву вікна виправити просто. Для цього треба скористатися ще одним важливим інструментом — редактором властивостей.
- Вибираємо в Object inspector MainWindow. Внизу праворуч, шукаємо властивість windowsTitle. Вписуємо бажану назву:



Написати код трішки важче. Приступимо.

- Почнемо з найпростішої дії — quit.
- Щоб створити обробник, йдемо до редактору дій, викликаємо контекстне меню для дії actionQuit, вибираэмо Go to slot –> triggered().
- До оголошення класу додасться:

```
private slots :
    void on_actionQuit_triggered();
```

а в .cpp файлі з'явиться заготовка для обробника:

```
void MainWindow::on_actionQuit_triggered()
{

}
```

Слоти, імена яких починаються з on_

- Імена слотів, які починаються з on_, інтерпретуються moc специфічно. Він проглядає список імен слотів, шукаючи такі, які підходять під шаблон: `on_objectName_signal`, і якщо зустрічає, тоді шукає об'єкт (вказівник) з іменем objectName та приєднує до цього слоту його сигнал з іменем signal.
- Якщо так називати свої слоти та об'єкти, зникає потреба явно викликати connect().
- Якщо ж писати імена слотів так, що парсер сприйматиме їх як "свої", але які при тому не відповідатимуть об'єктам і/або сигналам, нічого страшного не станеться — під час запуску програма буде видавати (за допомогою qWarning()) повідомлення типу: "No matching signal for...", та й все.
- Тобто, або дотримуйтеся описаної вище угоди, або не починайте імена слотів з on_.

Див. також `http://www.qtforum.org/article/20685/connectslotsbyname.html`.

Слот quit — 1

- Для того, щоб контролювати, чи не закривають програму із не збереженими даними, введемо дві змінні:

  bool new_file;
  bool file_changed

  із промовистими назвами, які в конструкторі встановлюються у false.

Слот quit — 2

Відповідний слот виглядає так:

```
void MainWindow::on_actionQuit_triggered()
{
    if (file_changed)
    {
        switch( askToSaveChanged(tr("Current changes are not saved.\
\nDo you really want to quit?")) )
        {
        case QMessageBox::Yes:
            QApplication :: instance()->quit();
            break;
        case QMessageBox::Save:
            emit ui->actionSave->triggered();
            break;
        }
    }else
    {
        QApplication :: instance()->quit();
    }
}
```

Слот quit — 3

- Цей слот перевіряє, чи поточний файл змінено (за встановлення `file_changed` відповідає інший код), якщо ні — завершує виконання.
- Інакше, за допомогою функції `askToSaveChanged` питається, чи зберегти зміни. Функція тривіальна, створює QMessageBox, викликає його та повертає вибір користувача. Вона надає наступні варіанти вибору: `QMessageBox::Yes`, `QMessageBox::Save`, `QMessageBox::Cancel`. Детальніше див. коди прикладу, `qtl06_ui_editor`.
- Якщо користувач відмовився зберігати — завершуємо нашу програму, інакше генеруємо сигнал triggered для дії (action) `actionSave`, яка створена для збереження — щоб її "інфраструктура" відпрацювала, і нам не довелося повторно реалізовувати логіку збереження. Cancel просто припиняє спробу завершити.
- Генерація сигналу здійснюється ще одним розширенням Qt (яким займається moc), командою emit:

```
emit ui->actionSave->triggered();
```

Візуальні віджети

Подивимося ще раз на код:

```
emit ui->actionSave->triggered();
```

- ui — вказівник на клас, що реалізовує візуальну частину інтерфейсу. Нагадаємо, що він генерується автоматично, з форми.
- actionSave — це назва дії, яку ми надали їй в редакторі форм (Qt Designer).
- Всі оголошені там компоненти (віджети та інші) доступні нам із заданими там іменами за допомогою коду типу `ui->componentName`.

Слот quit — 4

- Завершити програму можна ще одним способом — натиснувши хрестик у правому верхньому куті вікна (в деяких ОС він може бути не там).
- Перехопити такий метод завершення програми, щоб перевірити, чи все ми зберегли, можна, перевизначивши віртуальну функцію

  void closeEvent(QCloseEvent *bar);

- У ній просто активуємо наш обробник завершення програми:

```
void MainWindow::closeEvent(QCloseEvent *bar)
{
    emit ui->actionQuit->triggered();
    // Якщо ми потрапили сюди -- користувач вирішив
    // не завершувати програму.
    bar->ignore();
}
```

Оголошення класу головного вікна нашої програми — 1

```
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = 0);
    ~MainWindow();

private slots:
    //Всі види:
    void on_action ..... _triggered();

    // Слоти пошуку та заміни
    void onFind(const QString &str, QTextDocument::FindFlags ff);
    void onFindForReplace(const QString &str, QTextDocument::FindFlags ff);
    void onReplace(const QString &strFind, const QString &strReplace, QTex
    void onReplaceAll(const QString &strFind, const QString &strReplace, Q

private:
        ..........................
}
```

- Клас є нащадком QMainWindow, на початку містить традиційний для нащадків макрос Q_OBJECT.
- Далі йдуть конструктор і деструктор, котрі розглянемо пізніше.
- Секція слотів починається із слотів, згенерованих редактором форм.
- Після них йдуть чотири слота, які обслуговують пошук та заміну[2].

[2] Пошук та заміну розглянемо пізніше — це менш-більш незалежні частини програми, крім того, вони використовують ряд нових концепцій.

Оголошення класу головного вікна нашої програми — 2

```
private:
    QString app_name;

    void closeEvent(QCloseEvent *bar);

    void saveToFile(const QString& fileName);
    int askToSaveChanged(const QString& title);
    void setFileChangedStatus(bool fc=true);
    void setFileNewStatus(bool fc=true);
    void setFileName(const QString& fileName);
    bool new_file;
    bool file_changed;

    void SetupStatusbar();
QLabel* sbFilename, *sbFileSymbols, *sbFileCol, *sbFileRow, *sbFileChanged;

    QString curFileName;  QString filesMask;

    QScopedPointer<FindDialog> findDialog;
    QScopedPointer<ReplaceDialog> replaceDialog;
```

Коструктор

```
ui->setupUi(this);
setWindowIcon( QIcon(":/icons/app_icon.png") );
app_name = windowTitle( );
filesMask = tr( "Text Files (*.txt );; C++ Files (*.cpp *.h)");
SetupStatusbar();
connect( findDialog.data(), SIGNAL(find (....)), this, SLOT(onFind(...)));
connect( replaceDialog.data(), SIGNAL(find (...)), this, SLOT(onFindForRe
connect( replaceDialog.data(), SIGNAL(replace (...)), this, SLOT(onReplac
connect( replaceDialog.data(), SIGNAL(replaceAll (...)), this, SLOT(onRep
```

Сигнали та слоти пошуку розглянемо пізніше. Решта сигналів приєднуються автоматично, як описувалося раніше, тому в конструкторі явно їх не викликаємо.

Конструктор

- Конструюємо графічну частину:

  ui->setupUi(**this**);

- Встановлюємо іконку програми.

  setWindowIcon(QIcon(":/icons/app_icon.png"));

  Використано клас QIcon, детальніше про нього див. `http://qt-project.org/doc/qt-5/qicon.html`.
- Ім'я програми нам знадобиться в майбутньому.

  app_name = windowTitle( );

  Ми встановили його в редакторі форм, щоб не дублювати, отримуємо його методом windowTitle( ).
- Змінна filesMask містить маску файлів, з якими ми працюватимемо.
- На жаль, візуально налаштувати стрічку статусу поки неможливо, тому робимо це вручну, в функції SetupStatusbar().
- Слоти, згенеровані візуальним середовищем, під'єднані до відповідних сигналів автоматично. В коді ми додатково приєднуємо слоти, пов'язані з пошуком та заміною.

Стрічка статусу, функція SetupStatusbar() — 1

- Розташуємо на стрічці статусу наступні компоненти: Ім'я відкритого файлу, кількість символів у ньому, поточний рядок та стовпець та індикатор, чи було файл змінено. Для всіх значимих полів додамо tooltip-підказку.
- Ім'я відкритого файлу:

```
sbFilename = new QLabel("");
statusBar()->addWidget(sbFilename);
```

- Перманентне поле кількості символів:

```
sbFileSymbols = new QLabel("      ");
sbFileSymbols->setMinimumSize( sbFileSymbols->sizeHint() );
sbFileSymbols->setAlignment(Qt::AlignCenter);
sbFileSymbols->setToolTip(tr("Symbols in  file" ));
statusBar()->addPermanentWidget(sbFileSymbols);
```

  Зверніть увагу на виклик методу setToolTip().
- Решта полів додаємо аналогічно – див. код прикладу.

Деструктор

Так як для збереження діалогів (див. далі) скористалися розумними вказівниками, деструктор зовсім простенький:

```
MainWindow::~MainWindow()
{
    delete ui;
}
```

QTextEdit

Щоб розуміти, як працюють у роботі решта методів, слід розібратися, як функціює наш головний віджет, QTextEdit.

- Документація на нього знаходиться тут: `http://qt-project.org/doc/qt-4.8/qtextedit.html`. Клас цей дуже потужний, тому оглядатимемо лише важливі для нас його можливості та засоби.
- Призначення цього класу — вивід та редагування форматованого тексту.
- Для збереження форматування використовується підмножина HTML 4.0. Детальніше почитати, яка саме частина стандарту HTML підтримується, можна тут: `https://qt-project.org/doc/qt-5/richtext-html-subset.html`. Якщо потрібна більш потужна підтримка HTML, див. модуль QtWebKit.
- Якщо коротко, підтримуються всі основні форми виділення тексту — підкреслений, bold, italic, тощо, параграфи, включаючи вирівнювання, різні шрифти та кольори тексту, зображення, таблиці, списки, тощо. Ми, поки що, не використовуватимемо його просунуті можливості.
- Якщо текст не вміщається у виділений віджету простір, вміє відображати стрічки прокрутки.

З точки зору користувача

Клас QTextEdit підтримує менш-більш звичні засоби редагування тексту:

- Рух по тексту за допомогою клавіш із стрілками, Pg Up, Pg Down, Home, End, тощо.
- Вибір тексту, якщо переміщення здійснюється із натиснутим Shift. Вибір всього тексту — Ctrl-A.
- Стандартні шоткати редагування — Ctrl-C, Ctrl-V, Ctrl-X, Backspace, Del і т.д. і т.п.. Детальніше див. документацію.

- Слот

  void QTextEdit::setText (**const** QString & text)

  встановлює новий текст, викидаючи попередній. Формат тексту визначається автоматично. Щоб задати його примусово, можна використати методи-слоти setHtml() або setPlainText().

- Аналогічно, методи:

  QString toPlainText () **const**
  QString toHtml () **const**

  (а також застарілий метод text()) дозволяють подивитися на текст, збережений у віджеті.

- clear() очищає вміст віджету, викидаючи весь текст.

- Для маніпуляції текстом слід використовувати клас QTextCursor. Детальніше про нього далі. Для зручності, клас QTextEdit має методи маніпуляції вмісту, які самі вже звертаються до об'єкту типу QTextCursor, який відповідає поточному розташуванню курсора в тексті: insertHtml(), insertPlainText(), append(), paste(), moveCursor(), тощо.

- Отримати **копію** поточного курсору можна методом:

  QTextCursor QTextEdit::textCursor () **const**

  При тому, наприклад, insertPlainText є еквівалентний до:

  ```
  edit->textCursor().insertText( text );
  ```

- Щоб встановити курсор (наприклад, змінивши якимось чином поточний), слід скористатися setTextCursor().
- Щоб текст, де є курсор зараз, став видимим, можна використати метод:

  void QTextEdit:: ensureCursorVisible ()

Виділення

- Крім виділення, зробленого користувачем, існують програмні засоби створювати та маніпулювати виділенням.
- **void** QTextEdit::moveCursor (QTextCursor::MoveOperation operation, QTextCursor::MoveMode mode = QTextCursor::MoveAnchor)

  переміщає курсор, згідно вибраної операції MoveOperation (їх є кілька десятків, типу Left — на символ вліво, WordLeft – на слово вліво, Up – на лінійку вверх, EndOfWord – до кінця слова, і т.д., див. документацію.)
- Режим mode може мати два значення, MoveAnchor — переміщає якір разом із курсором, KeepAnchor — залишає його у вихідній позиції. При цьому, текст між новим і старим положенням курсору буде виділено!
- selectAll () виділяє весь текст.
- Метод find шукає передану стрічку, і якщо знаходить — виділяє її:

  bool QTextEdit::find (**const** QString & exp, QTextDocument::FindFlags options = 0)

  Опції пошуку, `options`, комбінується побітовим OR ("|"), з наступних варіантів: FindBackward, FindCaseSensitively, FindWholeWords, які є членами перерахування FindFlags з

Маніпуляції над виділенним текстом

- Слот `void QTextEdit::copy ()` копіює виділення в кишеньку.
- Слот `void QTextEdit::cut ()` видаляє видалення після копіювання його в кишеню.
- Слот `void QTextEdit::paste ()` вставляє текст з кишеньки, заміняючи виділення, якщо таке присутнє.
- Сигнал copyAvailable генерується, коли текст виділяється та коли знімається виділення. Аналогічно, сигнал selectionChanged () генерується за зміни виділення.

Керування форматом

- Не вдаючись в деталі, перерахую методи, які керують поточним форматом тексту: setFontItalic(), setFontWeight(), setFontUnderline(), setFontFamily(), setFontPointSize(), setTextColor(), setCurrentFont(). Вирівнювання поточного параграфу керується методом setAlignment(). Детальніше див. документацію.
- Перенос по словах керується методом setWordWrapMode().

Технічні деталі

- QTextEdit є нащадком QAbstractScrollArea. Цей клас потрібен будь-яким віджетам, яким потрібна прокрутка. Детальніше див. `http://qt-project.org/doc/qt-5/qabstractscrollarea.html`.
- Від нього, у свою чергу, успадковується QTextBrowser — компонента, яка не дає редагувати текст, але дозволяє його проглядати із додатковими засобами навігації. Детальніше див. `http://qt-project.org/doc/qt-5/qtextbrowser.html`.

QTextDocument — 1

- Документ, який відображає QTextEdit знаходиться в об'єкті класу QTextDocument (`http://qt-project.org/doc/qt-5/qtextdocument.html`).
- Отримати поточний документ можна таким методом:

  QTextDocument * QTextEdit::document () **const**

- Встановити новий:

  void QTextEdit::setDocument (QTextDocument * document)

  Увага, цей метод НЕ ЗАХОПЛЮЄ володіння QTextDocument, тому об'єкт класу QTextEdit не видалятиме його!
- QTextDocument генерує сигнал textChanged(), коли документ змінився. QTextEdit "ретранслює" його:

  void QTextEdit::textChanged () ;

QTextDocument — 2

- Аналогічно до textChanged та інших згадуваних ранішк, методи-слоти QTextDocument::undo та QTextDocument::redo теж доступні, для зручності, з QTextEdit:

  void QTextEdit::undo ();
  void QTextEdit::redo ();

- Слідкувати, чи наявне щось для undo/redo можна за допомогою сигналів undoAvailable та redoAvailable.
- Кількість символів у документі можна взнати за допомогою:

  int QTextDocument::characterCount() **const**

QTextCursor — 1

- Для маніпуляцій текстом QTextDocument (безпосередньо, чи через QTextEdit) використовується QTextCursor.
- Вище розглядалися методи, які QTextEdit "перенаправляє" QTextCursor, який відповідає поточному розташуванню курсору.
- Об'єкт класу QTextCursor, на далі — просто курсор, намагається поводитися так само, як і звичний видимий курсор текстових редакторів.
- Зокрема, він розташовується між літерами.
- Поточне розташування можна взнати за допомогою:

  int QTextCursor::position () **const**

  Вважається, що поточна літера — та що зразу **перед** курсором.
- Курсор також має так званий якір. Те, що знаходиться між курсором та його якорем, вважається виділеним.

  int QTextCursor::anchor () **const**

QTextCursor — 2

- Керувати положенням курсора можна відповідними методами, наприклад:

```
void QTextCursor:: setPosition  (  int  pos,
        MoveMode m = MoveAnchor );
bool QTextCursor::movePosition (  MoveOperation operation,
        MoveMode mode = MoveAnchor, int n = 1 );
```

  MoveMode нам вже відомий — він вибирає, чи переміщати якір разом із курсором. Зауважте, що movePosition, на відміну від аналогічного методу QTextEdit, курсор вміє переміщатися на вказану кількість вибраних MoveOperation кроків — літер, слів, рядків, тощо. Він також повертає true, якщо переміщення успішно відбулося.

- Отримати поточний стовпець та рядок курсору можна так:

```
int  QTextCursor::columnNumber () const
int  QTextCursor::blockNumber () const
```

  Слово блок в blockNumber стосується того, що у форматованому тексті символи кінця рядка розділяють абзаци, а розбиття на лінійки здійснюється автоматично.

QTextCursor — 3

- Отримати поточне виділення можна методами:

  int QTextCursor:: selectionStart () **const**;
  int QTextCursor::selectionEnd () **const**;

  Якщо не виділено нічого, вони повертають поточне положення курсору.
- Перевірити, чи взагалі є виділення, можна методом:

  bool QTextCursor::hasSelection () **const**;

- Зняти виділення:

  void QTextCursor:: clearSelection ();

QTextCursor — 4

- Курсор має багато інших методів — редагування (deleteChar(), deletePreviousChar(), removeSelectedText(), insertList(), insertTable() , insertImage(), insertFrame(), insertBlock(), insertText(), тощо), керування форматом (setCharFormat(), mergeCharFormat(), setBlockFormat(), mergeBlockFormat()), і т.д. і т.п.
- Детальніше ми їх тут не розглядатимемо. Так само, не розглядаються робота з більш просунутим форматуванням – абзацами, таблицями, списками.
- Детальніше див. документацію:
  - `http://qt-project.org/doc/qt-5/qtextedit.html` — QTextEdit.
  - `http://qt-project.org/doc/qt-5/qtextcursor.html` — QTextCursor.
  - `http://qt-project.org/doc/qt-5/qtextdocument.html` — QTextDocument.
  - `http://qt-project.org/doc/qt-5/qtextblock.html` — QTextBlock.
  - `http://qt-project.org/doc/qt-5/richtext.html` — Rich Text Processing tools.

Тепер, знаючи як функціює QTextEdit та його сімейство, можна знову повернутися до нашого коду.

- Слот, Який реагує на зміну документу:

```
void MainWindow::on_actionTextChanged_triggered(){
    setFileChangedStatus(true);
    sbFileSymbols->setNum(
        ui->textEdit->document()->characterCount()-1 );
}
```

- Вона викликає функцію, котра змінює вигляд програми, відображаючи – чи було файл змінено:

```
void MainWindow::setFileChangedStatus(bool fc){
    if(fc){
        file_changed = true;
        this->setWindowTitle(app_name + " (*)");
        sbFileChanged->setText("X");
    }else{
        file_changed = false;
        this->setWindowTitle(app_name);
        sbFileChanged->clear();
    }   }
```

- Якщо є зміни – до імені вікна додається зірочка і в стрічці статусу виставляється "прапорець"

- Копіювання:

```
void MainWindow::on_actionCopy_triggered(){
    ui->textEdit->copy();
}
```

- Вставка:

```
void MainWindow::on_actionPaste_triggered(){
    ui->textEdit->paste();
}
```

- Вирізання:

```
void MainWindow::on_actionCut_triggered(){
    ui->textEdit->cut();
}
```

- Видалення

```
void MainWindow::on_actionDelete_triggered(){
    QTextCursor curCursor = ui->textEdit->textCursor ();
    curCursor.removeSelectedText();
}
```

- В принципі, всі перераховані дії реалізовує, зав'язані на свої стандартні шоткати, сам QTextEdit, але корисно подивитися, як саме це робиться.

- Переміщення курсору теж потрібно відслідковувати, щоб змінювати поточний рядок та стовпець в рядку статусу:

```
void MainWindow::on_actionCursorMoved_triggered()
{
    QTextCursor curCursor = ui->textEdit->textCursor () ;
    sbFileCol->setNum( curCursor.positionInBlock()+1 );
    sbFileRow->setNum( curCursor.block().firstLineNumber()+1 );
}
```

Створення нового файлу

```
void MainWindow::on_actionNew_triggered()
{
    if (file_changed)
    {
        switch( askToSaveChanged(tr("Current changes are not saved."
                                    "\nDo you really want to throw out them?")) )
        {
        case QMessageBox::Cancel:
            return;
            break;
        case QMessageBox::Save:
            emit ui->actionSave->triggered();
            break;
        }
    }
    ui->textEdit->clear ();
    setFileNewStatus(true);
    setFileChangedStatus( false );
    setFileName(tr(""));
}
```

Слот, який створює новий файл, викликає ряд допоміжних функцій:

- askToSaveChanged() та setFileChangedStatus() ми вже розглядали раніше.
- setFileNewStatus() зовсім проста:

```
void MainWindow::setFileNewStatus(bool fc){
    // На відміну від setFileChangedStatus,
    // інших дій тут немає, тому if буде зайвим
    new_file=fc;
}
```

- setFileName() налаштовує вигляд та ім'я як для нового файлу:

```
curFileName = fileName;
QFileInfo curFI(curFileName);
sbFilename->setText( curFI.fileName() );
sbFilename->setToolTip( curFI.canonicalFilePath() );
```

Зауважте, що поля розміру документу та положення курсору ми не змінюємо — за ними "припильнує" on_actionTextChanged_triggered і on_actionCursorMoved_triggered.

QFile — 1

Наступна критично важлива частина текстового редактору — завантаження і збереження тексту. Перш ніж перейти до неї, слід ознайомитися із QFile, QTextStream та деякими допоміжними класами.

- QFile абстрагує роботу з файлами.
- Ім'я файлу цьому об'єкту можна передати в конструкторі, або під час виконання, методом setFileName():

```
QFile   file ("file.txt");
 file.setFileName("other_file.txt");
```

  Подивитися поточне ім'я можна методом file.fileName().
- В ролі сепаратора завжди використовується "/" і ніколи "\" — він заборонений. Під Windows конвертація під час звертання до ОС відбувається автоматично.
- Відкрити файл можна викликом методу open:

```
bool QFile::open(OpenMode mode);
```

  де mode задає режим. Якщо файл відкрився успішно, повертає true.

QFile — 2

- Бажаний режим отримується комбінуванням, за допомогою побітового OR ("|") таких прапорців, оголошених в enum QIODevice::OpenModeFlag: NotOpen, ReadOnly, WriteOnly, ReadWrite (= `ReadOnly | WriteOnly`), Append, Truncate, Text, Unbuffered:

```
if( ! file .open(QIODevice::ReadOnly | QIODevice::Text) )
{
    qCritical () << "Error opening file";
    return false;
}
```

Text означає, що символи кінця рядка будуть транслюватися при читанні в `'\n'`, а при записі у той, що прийнято локально, наприклад у пару `'\r\n'` під Windows. Призначення решти прапорців зрозуміле з контексту.

QFile — 3

- QFile є нащадком QFileDevice, який, у свою чергу успадковується від QIODevice. Тому працювати з файлом можна, використовуючи засоби останнього, зокрема (список не вичерпний):

```
qint64 QIODevice::read(char * data, qint64 maxSize);
QByteArray QIODevice::read(qint64 maxSize);
qint64 QIODevice::readLine(char * data, qint64 maxSize);
QByteArray QIODevice::readAll();
qint64 QIODevice::write(const char * data, qint64 maxSize);
qint64 QIODevice::write(const char * data);
qint64 QIODevice::write(const QByteArray & byteArray);

QByteArray QIODevice::read(qint64 maxSize);
bool QIODevice::putChar(char c);
void QIODevice::ungetChar(char c);
```

  де QByteArray — клас масиву байтів, детальніше див. документацію.

- Однак, часто зручніше працювати з ним, використовуючи потоки, QTextStream, про які — трішки пізніше.

QFile — 4

- Закрити файл можна викликом close():

```
file.close();
```

- Спорожнити буфери можна методом flush():

```
file.flush();
```

- Розмір файлу можна отримати методом size():

```
qint64 QFile::size() const;
```

- Отримати положення файлового вказівника:

```
qint64 QFileDevice::pos() const
```

- Встановити положення файлового вказівника:

```
bool QFileDevice::seek(qint64 pos);
```

- Можна змінити розмір файлу (якщо він збільшується — заповнюється нульовими байтами):

```
bool QFile::resize(qint64 sz)
```

Також є статична функція, яка змінює розмір файлу із переданим іменем:

```
bool QFile::resize(const QString & fileName, qint64 sz); [static]
```

QFile — 5

Наступні функції теж присутні в двох варіантах, другий варіант — статична функція, якій передається ім'я файлу.

- Перевірка існування файлу:

  bool QFile :: exists () **const**;
  bool QFile :: exists (**const** QString & fileName); [**static**]

- Видалити файл (закривається перед видаленням):

  bool QFile :: remove();
  bool QFile :: remove(**const** QString & fileName); [**static**]

- Перейменувати:

  bool QFile :: rename(**const** QString & newName);
  bool QFile :: rename(**const** QString & oldName, **const** QString & newName)

  Якщо файл із новим іменем вже існує – повертає false, не затираючи його.

- Якщо в процесі читання досягнуто кінець файлу, наступний метод повертає true:

  bool QFileDevice :: atEnd() **const**;

QFile — 6

Перераховане вище — далеко не все, що вміє QFile, але нам поки вистачить. Перш ніж рухатися далі, зверніть увагу на:

- QIODevice — `http://qt-project.org/doc/qt-5/qiodevice.html`, спільний предок для цілого сімейства класів вводу-виводу, наприклад QTcpSocket, QProcess (не рахуючи QFileDevice).
- QTemporaryFile — нащадок QFile для збереження тимчасових файлів, із гарантовано унікальним іменем, які автоматично видаляються по завершенню роботи з ним: `http://qt-project.org/doc/qt-5/qtemporaryfile.html`.
- QSaveFile, нащадок QFileDevice: `http://qt-project.org/doc/qt-5/qsavefile.html`. Реалізовує безпечне збереження даних — файл, що вже існує, перед збереженням перейменовується, якщо збереження пройшло успішно — видаляється. Завдяки такому механізму збій при записі не зіпсує вихідний документ.

Просунуті засоби роботи з файлами

На них ми, поки, теж не зупинятимемося, але ось:

- Клас QFileInfo надає портабельний доступ до інформації про файл. Поміж методів, робота з шляхами: absoluteDir(), absoluteFilePath(), absolutePath(), baseName(), canonicalFilePath(), canonicalPath(), completeBaseName(), completeSuffix(), dir(), fileName(), isAbsolute(), isNativePath(), isRelative(), isRoot(), makeAbsolute(), path(), suffix(), ; робота з властивостями файлу: created(), exists(), isDir(), isExecutable(), isFile(), isHidden(), isReadable(), isSymLink(), isWritable(), lastModified(), lastRead(), size(), і т.д. — декілька груп функцій опущено, детальніше див. `http://qt-project.org/doc/qt-5/qfileinfo.html`.
- QDir — робота з директоріями. Крім різноманітних маніпуляцій із шляхами, схожих на перераховані вище, типу absoluteFilePath(), вміє повертати спец-директорії: current(), root(), home(), temp(), (аналогічні виду currentPath() повертають результат у вигляді QString); вміє, що особливо цінно, виводити список вмісту директорії: **entryList()**. Детальніше див. `http://qt-project.org/doc/qt-5/qdir.html`.

Обидва розглянутих класи, як і сам QFile з компанією, вміють працювати з ресурсами Qt.

QTextStream — 1

Для зручності роботи з файлами можна скористатися потоковими класами, такими як QTextStream або QDataStream, схожими на потоки C++.

- QTextStream працює над текстовими даними.
- Він може працювати з QIODevice, QByteArray або QString (згадуємо std::stringstream зразу).
- Пов'язати потік із файлом:

```
QFile file ("output.txt");
if ( file.open(QFile::WriteOnly | QFile::Truncate)) {
    QTextStream out(&data);
      .......
}
```

можна, передавши йому адресу відповідного об'єкту. Крім виклику конструктора, потік із об'єктом можна пов'язати за допомогою setDevice() або setString().

- Всередині потік користується Unicode, перетворення кодувань здійснює автоматично, використовуючи QTextCodec. Детальніше див. документацію.

QTextStream — 2

- Читати і писати в потік можна за допомогою звичних операторів ”<<” і ”>>”:

  out << ”Result: ” << x;

- Має свої маніпулятори (список див. далі):

  out << qSetFieldWidth(10) << left << 3.141592 << endl;

  Як і для маніпуляторів стандартної бібліотеки, їм відповідають методи класу потоку:

  ```
  out.setFieldWidth(10);
  out.flush();
  ```

  які, в більшості випадків, менш зручні у використанні.

- Крім операторів ”<<” і ”>>”, можна читати за допомогою методів readLine() – по рядках, або readAll() – зразу все.
- **УВАГА!** Не варто читати дані з файлі і методами QFile і методами потоків одночасно! Або так, або так.

Детальніше див.
`http://qt-project.org/doc/qt-5/qtextstream.html`.

QTextStream — 3, маніпулятори

- setIntegerBase(N) встановлює основу системи числення, крім того є "фіксовані" його варіанти: bin, odct, dec, hex.
- setNumberFlags() дозволяє керувати виведенням чисел, приймаючи комбінацію аргументів типу QTextStream::NumberFlag: ShowBase, ForcePoint, ForceSign, UppercaseBase, UppercaseDigits. Є відповідні задані наперед маніпулятори без аргументів: showbase, forcesign, forcepoint, noshowbase, noforcesign, noforcepoint, uppercasebase, uppercasedigits, lowercasebase, lowercasedigits.
- setRealNumberNotation() керує виведенням чисел з рухомою крапкою, отримуючи аргумент типу QTextStream::RealNumberNotation: ScientificNotation, FixedNotation або SmartNotation. Відповідні маніпулятори без аргументів: fixed, scientific.
- Вирівнювання: left, right, center.
- Славнозвісний endl та його менш популярний кузен flush.
- ws, який наказує пропускати пробільні символи.
- reset відміняє всі зміни, зроблені маніпуляторами.
- Наступний маніпулятор встановлює ширину поля, але на відміну від std::setw(), діє, поки не відмінять:

  QTextStreamManipulator qSetFieldWidth (**int** width);

- Маніпулятори, які встановлюють символ для вирівнювання та точність для чисел з рухомою крапкою:

  QTextStreamManipulator qSetPadChar (QChar ch)
  QTextStreamManipulator qSetRealNumberPrecision (**int** precision)

QDataStream

- QDataStream працює із бінарними потоками.
- Робить це, незалежно від апаратної архітектури чи ОС, тому той самий файл з даними однаково читатиметься на різних платформах, не заважаючи, наприклад, на різний порядок байт в машинному слові.
- Реалізовано серіалізацію фундаментальних типів.
- З її використанням, реалізовано серіалізацію багатьох типів Qt, таких як QBrush, QColor, QDateTime, QFont, QPixmap, QString, QVariant, тощо та контейнерів Qt: QList, QLinkedList, QVector, QSet, QHash, QMap. Повний список можна побачити тут: `http://qt-project.org/doc/qt-5/datastreamformat.html`.
- Є можливість встановлення версій двійкових файлів.
- Детальніше див. `http://qt-project.org/doc/qt-5/qdatastream.html`.

Завантаження файлів

Нарешті можна перейти до того, як наш редактор читає файли:

```
void MainWindow::on_actionOpen_triggered()
{
    if (file_changed)
    {
        switch( askToSaveChanged(tr("Current changes are not saved."
                                    "\nDo you really want to throw out then
        {
        case QMessageBox::Cancel:
            return;
            break;
        case QMessageBox::Save:
            emit ui->actionSave->triggered();
            break;
        }
    }
        .....
```

Спочатку перевіряємо, чи поточний файл змінено. Якщо так — перепитуємося, що робити далі. Якщо користувач хоче зберегти зміни — генеруємо сигнал triggered() дії actionSave.

- Далі відкриваємо файловий діалог для вибору — що ж відкривати:

```
.....
setFileName(
    QFileDialog::getOpenFileName(this, tr("Open File"),
            QString(),  filesMask)
);
.....
```

наш метод setFileName ми вже розглядали. Діалогу передаємо нашу маску файлів, filesMask.

- Якщо користувач вибрав файл, намагаємося його відкрити:

```
if (!curFileName.isEmpty()) {

    QFile file(curFileName);
    if (!file.open(QIODevice::ReadOnly)) {
        QMessageBox::critical(this, tr("Error"),
            QString(tr("Could not open file %1").arg(curFileName)));
        return;
    }
```

- Якщо вдалося, пробуємо прочитати весь файл зразу:

```
    QTextStream data(&file);
    ui->textEdit->setText(data.readAll());
    setFileNewStatus(false);
    setFileChangedStatus(false);
    file.close();
}
```

після чого налаштовуємо наше візуальне представлення і закриваємо файл.

Збереження файлів

Збереження існує в більшій кількості варіантів, так як різних можливих ситуацій більше:

- "Save as": Питаємося, а куди зберегти, якщо користувач щось вибрав, викликаємо функцію, яка власне зберігає:

```
void MainWindow::on_actionSave_as_triggered()
{
    QString newFileName = QFileDialog::getSaveFileName(this,
        tr("Save File"), QString(), filesMask);
    if (!newFileName.isEmpty()) {
        saveToFile(newFileName);
        setFileName(newFileName);
    }
}
```

- На базі "Save as" зроблено просто збереження, "Save":

```
void MainWindow::on_actionSave_triggered()
{
    if (!curFileName.isEmpty() && !new_file ) {
        saveToFile(curFileName);
    }else{
        emit ui->actionSave_as->triggered();
    }
}
```

Якщо ім'я файлу вже вибране, зберігаємо в нього, згенерувавши відповідний сигнал. Інакше – просто зберігаємо.

- Робочою конячкою збереження, запису у файл є наступна функція:

```
void MainWindow::saveToFile(const QString& fileName){
    QFile file(fileName);
    if (!file.open(QIODevice::WriteOnly)) {
        QMessageBox::critical(this, tr("Error"), QString(tr("Could not
        emit ui->actionSave_as->triggered();
    } else {
        QTextStream stream(&file);
        stream << ui->textEdit->toPlainText();
        stream.flush();

        setFileNewStatus(false);
        setFileChangedStatus(false);

        file.close();
    }
}
```

Вона відкриває файл, якщо це вдалося — записує дані у вигляді "плоского" тексту, очищає буфер та встановлює стан "немає незмінених даних". Якщо відкрити файл не вдалося, пропонує ввести інше ім'я.

”Продовження буде...”

Ми ще не закінчили, однак, через розмір лекції та великий об’єм необхідного матеріалу, продовження — діалоги пошуку та заміни, розглядаються в наступній лекції.